High-Frequency Trading Order Book Benchmarking in C++20

Anthony Nguyen

Supervisor: Steve Pettifer

BSc (Hons) Computer Science

Department of Computer Science

UNIVERSITY OF MANCHESTER

April 13, 2026

Contents

Abstract **viii**

1 Introduction **1**
1.1 Context and Motivation 1
1.2 Problem Statement and Research Gap 1
1.3 Aim and Objectives 2
1.4 Research Hypotheses 2
1.5 Scope and Boundaries 3
1.6 Contributions Overview 3
1.7 Dissertation Structure 3

2 Background **4**
2.1 High-Frequency Trading and the Limit Order Book 4
2.2 Hardware Architecture 5
2.2.1 Cache Hierarchy and Memory Access 6
2.2.2 Branch Prediction 7
2.3 Data Structures for Order Books 7
2.3.1 Tree-Based Implementations 8
2.3.2 Flat Array Structures 8
2.3.3 Memory Pooling 9
2.4 Lock-Free Concurrent Queues 11
2.4.1 Exchange Architecture and Queue Modelling 11
2.4.2 SPSC and MPMC Implementations 11
2.4.3 Atomic Operations and Cache Coherence 13
2.4.4 The Standard Linux Network Stack 14
2.4.5 Kernel-Bypass Networking 16
2.5 Related Work 17

3 System Design **18**
3.1 Formal Requirements 18
3.1.1 Functional Requirements 19

3.1.2 Non-Functional Requirements 20
3.2 Design Goals and Constraints 20
3.3 High-Level Architecture 21
3.3.1 Architectural Pattern: Ports and Adapters 21
3.3.2 System Modules 21
3.4 The IOrderBook Interface and Polymorphism 23
3.5 Benchmarking Design: The Two-Mode Philosophy 23
3.6 Concurrency and Threading Strategy 24

4 Implementation **25**
4.1 Software Environment and Toolchain 25
4.2 Order Book Implementations 25
4.2.1 Standard Associative Map (Baseline) 26
4.2.2 Direct-Indexed Array (O(1) Access) 26
4.2.3 Hybrid "Hot-Cold" Vector-Map 27
4.2.4 Pool-Allocated Order Book (Slab Allocation) 27
4.3 The TCP Order Gateway 28
4.3.1 Protocol Specification and Parsing 28
4.3.2 The Loopback Simulation Model 29
4.4 High-Performance Concurrency 30
4.4.1 Wait-Free SPSC Queue 30
4.5 Low-Level Performance Utilities 30
4.5.1 Nanosecond Precision Timing 30
4.5.2 Thread-to-Core Affinity (CPU Pinning) 31
4.6 Automation and Data Collection 31

5 Testing and Evaluation **32**
5.1 Experimental Methodology 32
5.2 Functional Validation 33
5.2.1 Core Logic and Semantic Parity 33
5.2.2 System Pipeline and Integration 35
5.2.3 Scaling and Stress Testing 35
5.2.4 Gateway Simulation Boundaries and Validation 37
5.3 Throughput Results 37
5.4 Latency Analysis - Direct Mode 38
5.4.1 Scenario-Specific Latency Trends 39
5.4.2 Architectural Mechanisms: Pool and Hybrid 42
5.4.3 Tail Latency Distribution and Determinism 45
5.5 Gateway Mode - End-to-End Decomposition 47

5.6 MPSC Thread Contention 49
5.7 Summary of Evaluation against Requirements 51
5.8 Threats to Validity 51

6 Conclusions **53**
6.1 Summary of Contributions 53
6.2 Answers to Research Hypotheses 53
6.3 Critical Reflection 54
6.4 Limitations 55
6.5 Practical Recommendations 55
6.6 Future Work 56
6.7 Closing Statement 56

A LockFreeQueue Implementation **60**

B ArrayOrderBook addOrder **61**

C Matching Engine Timestamp Decomposition **62**

D FIX Incremental Parser **63**

E PoolOrderBook Match Branching **64**

List of Figures

2.1 A stylized Limit Order Book demonstrating price-time priority grouping. The bid (buyers) and ask (sellers) are separated by the spread, establishing the mid-price. 5

2.2 CPU memory hierarchy illustrating the explicit size-latency tradeoff. Faster layers are closer to the execution core but exponentially smaller in capacity. 6

2.3 A chronological textbook view of a pipeline flush across discrete Clock Cycles (CC). When the branch (I_2) evaluates its condition at the Execute phase (CC4) and discovers a misprediction, all speculatively fetched incoming instructions (I_3, I_4) are forcefully discarded. The pipeline violently stalls until the correct target instructions (I_3') are fetched and advance through the queue. 8

2.4 Visual comparison of traditional heap allocation versus a Memory Pool. Dynamic allocations mathematically fragment across the heap as orders are rapidly added and deleted, forcing the CPU to fetch disjointed cache lines ("pointer chasing"). A Memory Pool allocates one massive contiguous block upfront, ensuring subsequent active object fetches naturally stream into the L1 hardware cache. 10

2.5 Simplified model of exchange message ingress. Packets arrive concurrently from multiple trading clients and are serialised by the Exchange NIC, which applies hardware timestamps and passes messages through an arbiter to enforce strict time-priority ordering. The resulting ordered stream is pushed into a Gateway Queue, from which the single Matching Engine thread consumes sequentially. 11

2.6 A lock-free Single-Producer Single-Consumer (SPSC) ring buffer. The producer actively writes to the Head pointer while the consumer asynchronously trails behind reading from the Tail pointer. Because the two threads concurrently modify distinct atomic variables, they operate entirely lock-free. So long as the underlying logic utilizes safe modulo arithmetic to wrap edges, the system remains infinitely scalable for isolated threads. 12

2.7 Multi-Producer Multi-Consumer (MPMC) queue contention model. Multiple concurrent producer threads (one per trading client) independently push order messages into the shared `moodycamel::ConcurrentQueue`. Atomic Head and Tail indices coordinate access without OS-level locking. A single consumer thread (the Matching Engine) dequeues messages sequentially, allowing the system to measure throughput degradation as producer count increases. 12
2.8 Structure of a FIX 4.2 "New Order Single" (MsgType=D) message. The protocol uses a human-readable tag-value encoding where fields are separated by the non-printable SOH character. While flexible, parsing this repeating sequence of ASCII characters induces significantly higher CPU overhead compared to binary protocols like SBE or FAST. 14
2.9 The standard Linux kernel network ingress pipeline. A packet must traverse multiple layers of abstraction—including DMA transfers, hardware interrupts, and TCP/IP stack logic—before reaching the application. Each stage, particularly the transition from kernel to user space via system calls, introduces non-deterministic delays that manifest as microsecond-scale jitter in HFT environments. 15
2.10 A comparison between the standard Linux networking stack and kernel-bypass technologies such as DPDK. While the standard stack requires the kernel to mediate packet delivery—introducing unpredictable context switches—kernel-bypass allows a user-space thread to poll the NIC hardware directly. This eliminates operating system overhead and provides the nanosecond-scale determinism required for HFT execution. 16

3.1 Data-Flow Overview: Pipeline stages from generation to telemetry. . . . 22
3.2 Hexagonal Architecture (Ports and Adapters): Clear separation between the matching domain and interchangeable adapters. 22

5.1 Mixed-scenario profile validation plot. The left panel confirms the 55/30/15 distribution across near-spread, deep, and extreme price levels, while the right panel verifies the 18% cancellation pressure. This validates the generator's output against its theoretical configuration (NFR-1, NFR-9). 35
5.2 Full System Architecture: From Gateway Client to Matching Engine . . . 36
5.3 Direct-mode mean latency heatmap across scenarios and order book implementations. Array is the fastest implementation in six of seven scenarios, while hybrid is narrowly fastest in tight_spread. 40

5.4 Direct-mode relative slowdown versus scenario winner (excluding vector). By removing the vector outlier, the performance nuances between the competitive non-vector implementations (Array, Hybrid, Pool, and Map) are preserved in the colormap scale. 41
5.5 Direct-mode standard-deviation heatmap (excluding vector). Lower values indicate tighter run-to-run stability, providing variance evidence alongside mean latency for NFR-6. 42
5.6 PoolOrderBook Matching Logic: Divergent Execution Paths 43
5.7 Cost decomposition of the `mixed` scenario (direct mode). The comparison highlights the Hybrid book's advantage in lookups and the Pool's match-processing bottleneck. 44
5.8 Tail inflation map using P99-to-mean ratio in direct mode. Several map and vector cases exhibit elevated tail amplification relative to mean latency, indicating sensitivity to jitter and/or burst conditions. 47
5.9 End-to-End Telemetry Checkpoints. The pipeline is instrumented with four precise timestamps to isolate transport, queuing, and matching latencies. 48
5.10 Dense_full gateway decomposition (network, queue, engine) for non-vector implementations. The vector row is omitted in this comparative view because its dense_full queue and network outlier compresses the non-vector bars. 48
5.11 Gateway latency by scenario across all implementations. The plot shows mean end-to-end latency patterns only; the masking claim is supported by the decomposition in Table 5.6. 49
5.12 Throughput scaling under varying producer thread counts in MPSC mode. Identifies the contention point where the single consumer matching engine becomes the bottleneck. 50
5.13 Latency breakdown showing how queue wait times grow as more threads are added, while engine matching time stays the same. 50

List of Tables

3.1 Functional Requirements (FR) 19
3.2 Non-Functional Requirements (NFR) 20

5.1 Code Coverage Results for Core Order Book Logic 34
5.2 Mapping of FIX 4.2 Application Tags to Internal C++ Order Structures 36
5.3 Mean Throughput Across All Implementations in Direct and Gateway Mode 38
5.4 Mean Operation Latency in Direct Mode (nanoseconds) 44
5.5 P99 Tail Latency in Direct Mode (nanoseconds) 46
5.6 End-to-End Latency Decomposition in Gateway Mode (nanoseconds), averaged across all seven scenarios 48

Abstract

A C++20 exchange infrastructure is presented for benchmarking five order book data structure strategies across direct and gateway operating modes. The framework contains a TCP order gateway, an incremental FIX parser, a lock-free single-producer single-consumer queue, and a single-threaded matching engine connected to interchangeable order book implementations. On the canonical pinned-run dataset (2026-03-26), direct-indexed array access is the strongest direct-mode implementation in six of seven scenarios, with mixed-scenario mean latency of 210.72 ns and dense-full mean latency of 213.44 ns. In contrast, sorted-vector insertion degrades sharply in dense books, reaching 16082.10 ns in dense-full direct mode (75.3x slower than array). Gateway-mode latency is dominated by network and transport overhead for non-vector rows (median network share 98.7%), which reduces the visible contribution of internal order-book differences to end-to-end timing. The evidence supports two bounded conclusions. First, data-structure execution path and memory-access behavior are the dominant factors when algorithmic work is isolated. Second, in end-to-end gateway operation, system-level bottlenecks can mask data-structure gains unless transport and parsing overhead are reduced. The project contributes a reproducible benchmark harness, scenario-driven workload generation, and decomposed network/queue/engine timing suitable for future experiments on kernel-bypass networking and multi-producer ingestion.

Chapter 1

Introduction

1.1 Context and Motivation

In finance, one domain naturally sits at the bleeding edge of software performance: high-frequency trading (HFT), where firms compete to execute orders at the lowest possible latency (Aldridge, 2013). In this context, the order book is one of the most important components, because it maintains the state of buy and sell orders for each instrument and sits on the critical processing path (Gould et al., 2013). The design of the order book can therefore directly affect execution quality and overall system behaviour.

Given this dependence on low-latency software, understanding the performance implications of different order-book designs is important for both researchers and practitioners. This dissertation evaluates and compares order-book data structures in a realistic benchmarking environment. Detailed technical definitions are provided in Chapter 2; this chapter focuses on the research problem, objectives, and scope of the study.

1.2 Problem Statement and Research Gap

Although there has been significant work on order-book data structures, much of industry practice remains opaque because implementation details are commercially sensitive. Any advantage gained from an optimised order-book design can be worth substantial financial value, so firms rarely disclose architecture decisions in full detail. This creates a useful role for academic benchmarking: transparent, reproducible evaluation under controlled conditions.

Another gap is methodological. Many comparisons emphasise isolated algorithmic performance but do not capture full end-to-end latency, where transport and system overhead can dominate. This dissertation addresses that gap with a two-mode approach: direct mode to isolate order-book behaviour, and gateway mode to evaluate the complete software path from network ingress to matching-engine processing.

1.3 Aim and Objectives

This dissertation aims to design, implement, and evaluate a C++20 exchange infrastructure for benchmarking order book data structures in high-frequency trading environments. The specific objectives are as follows:

- Define a modular benchmarking framework with interchangeable order-book implementations under a common interface.
- Implement and validate five order-book variants (map, array, vector, pool, and hybrid) with semantic parity across shared test scenarios.
- Evaluate performance in direct mode and gateway mode to separate algorithmic behaviour from end-to-end system overhead.
- Decompose gateway latency into network latency, queue latency, and engine latency to localise bottlenecks.
- Interpret observed differences using data-structure mechanics and hardware-aware reasoning grounded in measured evidence.

To evaluate these objectives, the study uses direct mode and gateway mode, and reports mean latency, P99 latency, throughput, and component-level decomposition of network latency, queue latency, and engine latency against H1–H3.

1.4 Research Hypotheses

The evaluation tests three hypotheses that connect algorithmic structure, allocator strategy, and systems overhead:

- **H1 (Data Structure and Cache Locality):** index-based order-book layouts are expected to outperform pointer-heavy alternatives in isolated operation.
- **H2 (Memory Allocation Limits):** memory pooling is expected to reduce selected allocation costs but not remove the fundamental structural penalties of slower layouts.
- **H3 (System-Level Masking):** in gateway operation, network and queue overheads are expected to compress or hide many internal data-structure performance differences.

For clarity, H1 is treated as supported when index-based designs consistently outperform pointer-heavy designs in direct mode across the core scenarios; H2 is treated

as supported when pooling improves selected allocation-heavy cases without eliminating structural penalties across mixed workloads; and H3 is treated as supported when gateway decomposition shows network and queue costs dominating engine cost, with compressed cross-implementation differences.

These hypotheses and acceptance criteria are defined formally in Chapter 5.

1.5 Scope and Boundaries

This dissertation focuses on software-path benchmarking within a controlled host environment, including direct and gateway execution modes, multiple workload profiles, and comparative analysis across five order-book implementations. The scope includes functional validation, throughput and latency measurement, and component-level decomposition of gateway costs. The scope does not include full physical network-path modelling (for example switch traversal and NIC hardware contention), and absolute latency values should therefore be interpreted as environment-bounded rather than universally transferable.

1.6 Contributions Overview

The dissertation makes four contributions. First, it presents a modular, extensible C++20 exchange framework with interchangeable order-book implementations and a TCP gateway path for end-to-end benchmarking. Second, it provides a reproducible benchmarking suite that separates direct-mode and gateway-mode behaviour while decomposing latency and throughput across pipeline components. Third, it reports empirical evidence on the trade-offs between cache-friendly indexing and pointer-based alternatives, including the bounded benefit profile of memory pooling. Fourth, it demonstrates a systems-level result: in gateway operation, network and queue overheads can dominate total latency and mask differences that are visible in isolated mode.

1.7 Dissertation Structure

Chapter 2 reviews the technical background required for the study, including market microstructure and hardware-aware performance considerations. Chapter 3 presents the architectural design, requirements framework, and benchmarking philosophy. Chapter 4 details implementation choices for the exchange pipeline and order-book variants. Chapter 5 reports functional validation and performance results against the research hypotheses. Finally, Chapter 6 summarises findings, limitations, and future work.

Chapter 2

Background

2.1 High-Frequency Trading and the Limit Order Book

High-Frequency Trading (HFT) is a specialized form of algorithmic trading that executes orders at millisecond or microsecond timescales. HFT strategies are characterized by high order-to-trade ratios, exceedingly short holding periods, and a reliance on co-located servers running directly adjacent to exchange matching engines (Aldridge, 2013). Leading proprietary trading firms, such as Optiver and Citadel Securities, secure direct market data feeds to bypass the latency overhead of standard retail brokers. These firms leverage their ultra-low latency infrastructure to execute a variety of latency-sensitive strategies (Menkveld, 2013). This market-design and speed interaction is also synthesised at the overall market-structure level by O'Hara (2015). This includes spread capture (acting as a market maker by buying at the bid and selling at the ask), avoiding adverse selection (canceling stale quotes before they are traded against by faster participants), and statistical arbitrage (exploiting fleeting price discrepancies across multiple exchange venues). As Avellaneda and Stoikov (2008) formalise, these strategies depend directly on how quickly the limit order book can be updated and acted upon, because even small delays can erode spread capture and increase adverse-selection risk.

A core technological component of this ecosystem is the Limit Order Book (LOB), illustrated in Figure 2.1. The LOB is a centralized electronic record of all active limit orders - instructions to buy or sell a specific quantity of an asset at a specified price or better (Gould et al., 2013). These orders are grouped by price levels and divided into two sides: the *bid* (buyers) and the *ask* or *offer* (sellers). The highest price a buyer is willing to pay is the best bid, while the lowest price a seller will accept is the best ask. The numerical difference between these two values is known as the spread (visually separating the two sides in Figure 2.1), which acts as a primary indicator of market liquidity and volatility. The midpoint between the best bid and best ask provides a theoretical "fair value" for

the asset at any given microsecond, while strategic order submission and cancellation behaviour that drives spread and depth evolution is formalised by Rosu (2009).

	Ask Price	Total Size	
Ask (Sellers)	100.03	50	
	100.02	120	
	100.01	400	← **Best Ask**

Spread = 0.02 | ***Mid-price = 100.00***

	Bid Price	Total Size	
Bid (Buyers)	99.99	250	← **Best Bid**
	99.98	600	
	99.97	150	

Figure 2.1: A stylized Limit Order Book demonstrating price-time priority grouping. The bid (buyers) and ask (sellers) are separated by the spread, establishing the mid-price.

While the official limit order book is maintained by the exchange matching engine, trading firms are responsible for reconstructing their own internal "shadow" books. Exchanges broadcast market events via raw binary protocols like ITCH or FIX, and quant firms must rapidly parse these feeds to rebuild the current book state in their own local memory. The performance of an HFT algorithm depends entirely on how quickly this internal shadow book can process events, making the underlying data structures and software architecture absolutely critical.

2.2 Hardware Architecture

To achieve true ultra-low latency, trading systems must be designed to cater directly to the physical realities of the underlying microprocessor. This approach is widely known as Mechanical Sympathy (Fowler, 2011b). The core philosophy dictates that understanding the hardware architecture of a machine enables software to operate in harmony with it, unlocking maximum potential throughput. Rather than treating the CPU as an abstract resource, HFT software treats hardware features - such as cache hierarchies and processor pipelines - as first-class design constraints.

2.2.1 Cache Hierarchy and Memory Access

The cache is arguably the most critical hardware component to optimise for when striving for ultra-low latency. It relies on the concepts of temporal and spatial locality to reduce memory operation times. Temporal locality dictates that recently accessed memory locations are highly likely to be accessed again in the near future. Spatial locality assumes that memory locations physically adjacent to a recently accessed address will also be needed soon. This behavior closely aligns with the Pareto principle, where 80% of memory accesses are typically concentrated within just 20% of an application's data.

To capitalize on this, the memory subsystem is organized into a rigid hierarchical structure, as visualized in Figure 2.2. The L1 cache is the smallest storage layer but resides physically closest to the processor core, offering extremely fast access times of roughly 1 - 2 nanoseconds. Subsequent caches (L2 and L3) are progressively larger but sit further away, incurring proportionately longer fetch times. At the bottom of this hierarchy lies the main system memory (RAM). Fetching data from main memory is significantly slower relative to the cache, routinely taking over 100 nanoseconds and stalling the CPU pipeline entirely (Patterson and Hennessy, 2013).

When the CPU fetches data from memory, it does not retrieve individual bytes; instead, it loads contiguous blocks known as *cache lines* (typically 64 bytes). This mechanism perfectly leverages spatial locality, ensuring that contiguous data structures - like flat arrays - can be sequentially processed with minimal misses. Conversely, non-contiguous allocations (such as pointer chasing in tree structures) break spatial locality, forcing the CPU to continuously stall while waiting for slow main memory fetches.

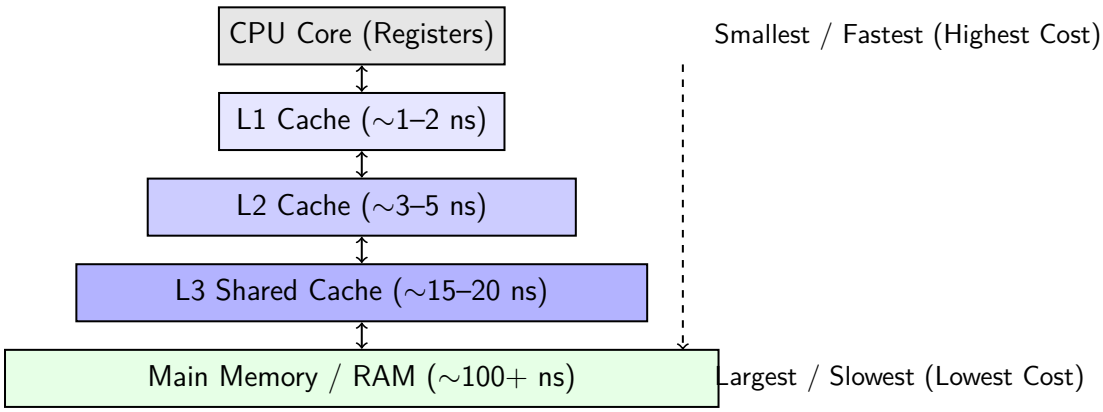

Figure 2.2: CPU memory hierarchy illustrating the explicit size-latency tradeoff. Faster layers are closer to the execution core but exponentially smaller in capacity.

2.2.2 Branch Prediction

Modern CPUs process computation through a specific sequence of stages known as the instruction pipeline, conceptualized in Figure 2.3. While architectures vary, a classic pipeline consists of four fundamental stages: Fetch, Decode, Execute, and Write-back (Patterson and Hennessy, 2013). When the CPU encounters a conditional branch (an `if-else` statement) during the Fetch phase, it does not actually know which path to take until the Execute phase resolves the condition. At this decision point, the CPU has two choices: wait for the execution to finish, or guess the outcome. Waiting would stall the pipeline completely, wasting upward of 15 to 20 clock cycles while the CPU sits idle doing nothing.

To avoid this massive waste of computational time, CPUs rely on a mechanism called a branch predictor. Instead of waiting, the branch predictor guesses which path the program will take and immediately begins speculatively fetching and decoding those instructions. If the guess is correct, execution continues seamlessly with zero wasted cycles. However, if the predictor guesses incorrectly, the CPU must flush all the speculative, half-finished work from the pipeline and restart fetching from the correct address. This pipeline flush incurs a severe penalty, typically costing modern architectures like the AMD Zen 4 between 15 and 20 cycles of completely lost time (Fog, 2025).

Modern predictors, such as the TAGE (TAgged GEometric) or perceptron-based predictors built into current AMD and Intel chips, are incredibly sophisticated and routinely achieve over 95% accuracy (Seznec and Michaud, 2006). Mathematically, the advantage is clear: assuming a 15-cycle misprediction penalty, a 95% accurate predictor yields an average penalty of just 0.75 cycles per branch ($0.95 \times 0 + 0.05 \times 15$), vastly outperforming a strategy of always stalling for 15 cycles.

However, despite this impressive average-case mathematical advantage, unpredictable branches are highly detrimental to ultra-low latency systems. In HFT, average performance is secondary to worst-case tail latency (the outermost response-time percentiles, such as the 99.9th percentile, which dictate the absolute maximum time a system takes to react under stress) (Dean and Barroso, 2013). If complex business logic (such as an unpredictable conditional order fill) resides on the critical path, the inevitable 5% of mispredictions will cause unpredictable, nanosecond-scale execution spikes. Removing unpredictable branches entirely is therefore a primary engineering goal to ensure deterministic, low-variance execution.

2.3 Data Structures for Order Books

The choice of underlying data structure fundamentally dictates the baseline performance of any limit order book. While theoretical algorithmic complexity provides a mathemat-

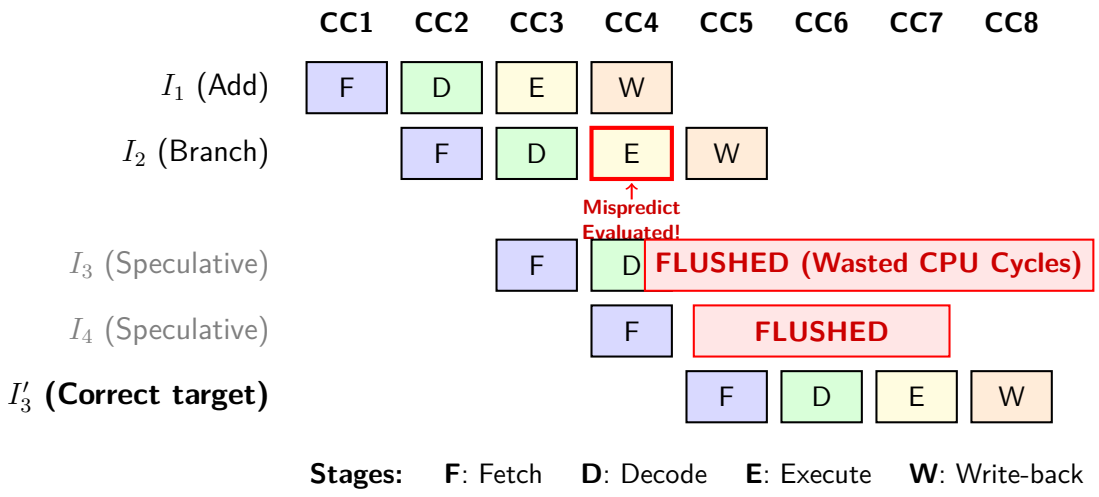

Figure 2.3: A chronological textbook view of a pipeline flush across discrete Clock Cycles (CC). When the branch (I_2) evaluates its condition at the Execute phase (CC4) and discovers a misprediction, all speculatively fetched incoming instructions (I_3, I_4) are forcefully discarded. The pipeline violently stalls until the correct target instructions (I'_3) are fetched and advance through the queue.

ical framework, the physical realities of modern hardware - namely the cache hierarchy and branch predictor - often subvert classical computer science assumptions.

2.3.1 Tree-Based Implementations

Historically, many order books have been implemented using tree-based structures such as Red-Black trees (often conceptually exposed as `std::map` in C++) to manage price levels. These structures offer a theoretical $\mathcal{O}(\log N)$ time complexity for insertions, deletions, and lookups. Their primary advantage is implementation simplicity, as a map intuitively models the domain of a limit order book by directly associating a price level (the key) with a queue of resident orders (the value). Furthermore, trees dynamically and gracefully handle massive, arbitrary price leaps without requiring predefined price boundaries. However, tree nodes are invariably allocated individually on the dynamic heap. As the order book processes thousands of intertwined updates, these nodes become fragmented across disparate locations in main memory. Consequently, traversing a tree to find a specific price level results in rapid, unpredictable pointer chasing. Each pointer dereference frequently triggers a cache miss, forcing the CPU to repeatedly stall while waiting for main memory fetches.

2.3.2 Flat Array Structures

To explicitly combat heap fragmentation, modern HFT architectures favor contiguous, flat structures such as pre-allocated arrays or vectors. Although inserting an element into

a sorted array incurs an $\mathcal{O}(N)$ worst-case complexity (due to shifting elements to maintain chronological order), this metric is theoretically inferior to a tree's $\mathcal{O}(\log N)$. However, because arrays are strictly contiguous in memory, traversing them allows the CPU's hardware prefetcher to eagerly load subsequent elements into the L1 cache before they are even requested (Patterson and Hennessy, 2013). For the relatively small, localized working sets typical of an active limit order book, the sheer speed of contiguous localized memory scans vastly outweighs the algorithmic overhead. In practice, an $\mathcal{O}(N)$ cache-friendly array scan routinely outperforms an $\mathcal{O}(\log N)$ tree traversal for latency-critical workloads. The severe trade-off for this speed is architectural rigidity: to achieve true $\mathcal{O}(1)$ direct array indexing based on price, the book must strictly map discrete *fixed-tick* price intervals to precise internal array offsets, sacrificing the dynamic flexibility of a tree.

2.3.3 Memory Pooling

The final structural consideration is how memory is allocated during live trading. Standard dynamic allocation functions like `std::malloc` or the `new` operator require the operating system to search for free heap space and occasionally request more memory pages from the kernel. This process requires context switching and is inherently non-deterministic. Executing a dynamic allocation on the critical trading path is therefore fatal to predictability because it randomly introduces severe microsecond-scale execution spikes.

As illustrated in Figure 2.4, to bypass the operating system entirely during the trading day, high-performance systems use Memory Pools - large contiguous blocks of memory allocated entirely during system startup. Typically, this is achieved by instructing the OS to persistently map a massive, pre-sized memory region (commonly via the POSIX `mmap` system call) capable of securely holding millions of queued objects without exhausting system bounds (Love, 2013). At runtime, inserting a new order simply claims a pre-allocated object from the local pool via a fast pointer bump, guaranteeing deterministic, $\mathcal{O}(1)$ nanosecond execution.

From a software engineering perspective, this profound architectural change is encapsulated behind standard interfaces. The memory pool acts as a drop-in replacement, exposing the same high-level API methods (e.g., `insert`, `erase`) as a traditional `std::map`, allowing the trading engine to remain agnostic to the underlying memory mechanics. This, however, shifts the burden entirely onto the developer, mandating manual capacity management and the implementation of custom logic to handle aggressive, unforeseen out-of-memory pool exhaustions.

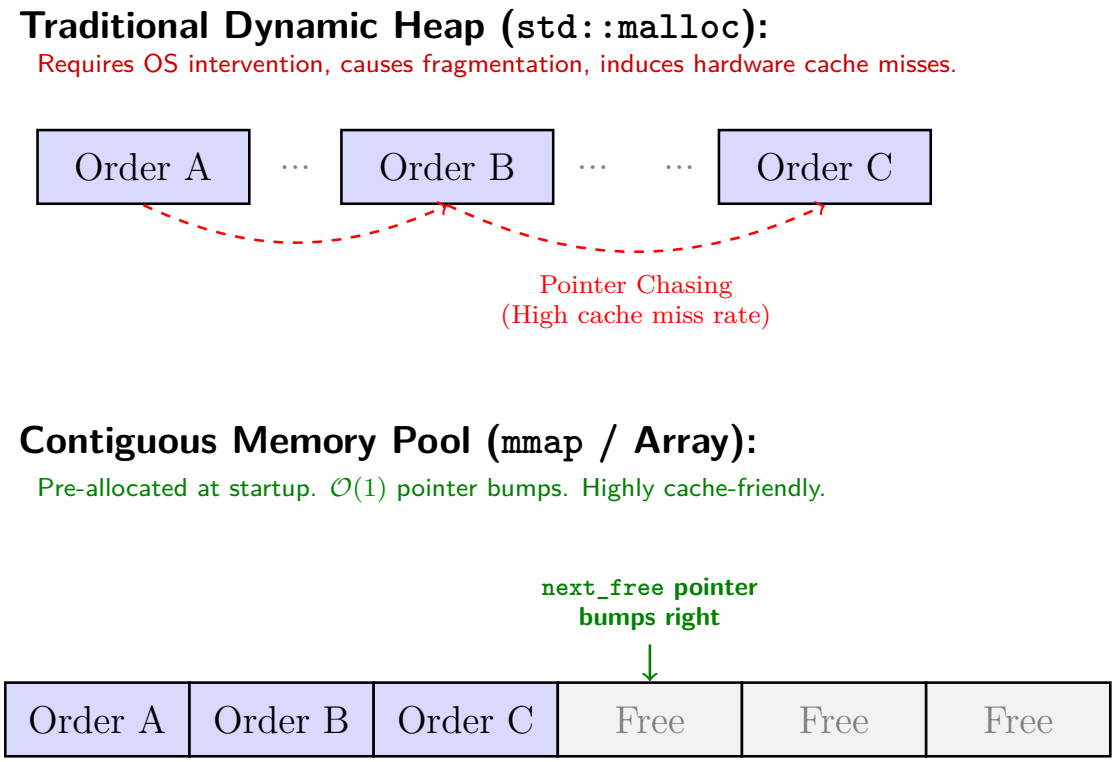

Figure 2.4: Visual comparison of traditional heap allocation versus a Memory Pool. Dynamic allocations mathematically fragment across the heap as orders are rapidly added and deleted, forcing the CPU to fetch disjointed cache lines ("pointer chasing"). A Memory Pool allocates one massive contiguous block upfront, ensuring subsequent active object fetches naturally stream into the L1 hardware cache.

2.4 Lock-Free Concurrent Queues

2.4.1 Exchange Architecture and Queue Modelling

In financial markets, trading firms transmit orders to exchanges across high-speed optical networks. Upon ingress, an exchange must safely and deterministically sequence these concurrent, intertwining arriving messages to enforce absolute chronological fairness. Physical exchanges achieve this sequencing via hardware - performing nanosecond packet timestamping directly on the ingress Network Interface Card (NIC) before funneling messages through a hardware-level arbiter (Harris, 2002). However, to stress-test the isolated performance algorithms of the matching engine without requiring proprietary networking hardware, our benchmark methodology strictly models this exchange boundary as a system of concurrent software queues. Network-facing threads act as producers that push validated messages into a gateway queue. The matching engine acts as the sole consumer, polling the queue to execute orders sequentially.

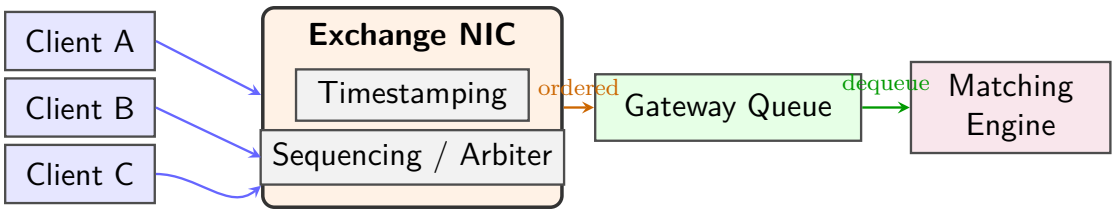

Figure 2.5: Simplified model of exchange message ingress. Packets arrive concurrently from multiple trading clients and are serialised by the Exchange NIC, which applies hardware timestamps and passes messages through an arbiter to enforce strict time-priority ordering. The resulting ordered stream is pushed into a Gateway Queue, from which the single Matching Engine thread consumes sequentially.

2.4.2 SPSC and MPMC Implementations

To evaluate this hand-off boundary, the architecture must forbid the use of operating-system locks. For a 1-to-1 thread mapping (one network thread, one matching engine), a custom Single-Producer Single-Consumer (SPSC) ring buffer is the natural model. As illustrated in Figure 2.6, an SPSC queue coordinates thread access using two entirely independent atomic variables: a *Head* index pushed exclusively by the producer, and a *Tail* index strictly advanced by the consumer.

To validate gateway scalability against concurrent trading clients, the benchmarking architecture must support Many-to-One messaging. For multi-client configurations, the system integrates `moodycamel::ConcurrentQueue` (Desrochers, 2013). This Multi-Producer Multi-Consumer (MPMC) structure allows the evaluation suite to simulate gateway contention from multiple concurrent threads and observe how throughput scales with producer count.

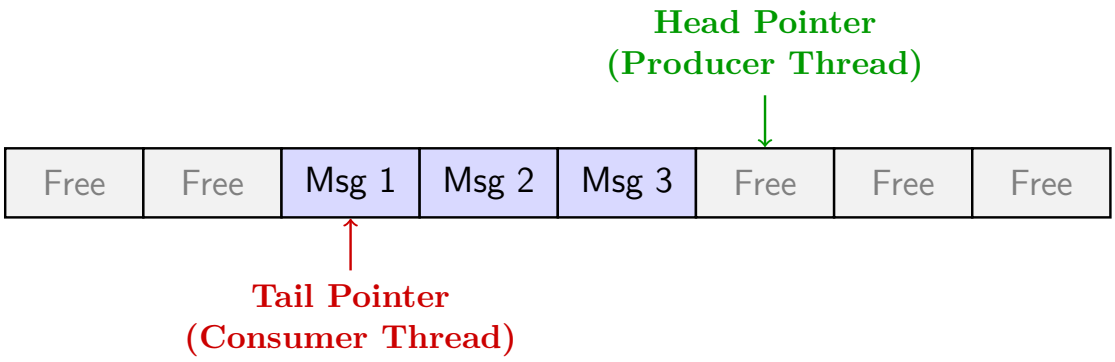

Figure 2.6: A lock-free Single-Producer Single-Consumer (SPSC) ring buffer. The producer actively writes to the Head pointer while the consumer asynchronously trails behind reading from the Tail pointer. Because the two threads concurrently modify distinct atomic variables, they operate entirely lock-free. So long as the underlying logic utilizes safe modulo arithmetic to wrap edges, the system remains infinitely scalable for isolated threads.

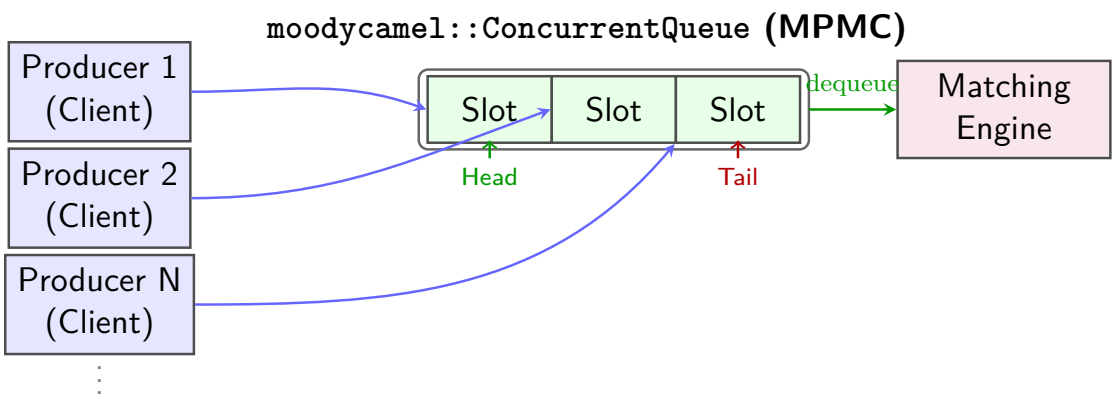

Figure 2.7: Multi-Producer Multi-Consumer (MPMC) queue contention model. Multiple concurrent producer threads (one per trading client) independently push order messages into the shared `moodycamel::ConcurrentQueue`. Atomic Head and Tail indices coordinate access without OS-level locking. A single consumer thread (the Matching Engine) dequeues messages sequentially, allowing the system to measure throughput degradation as producer count increases.

2.4.3 Atomic Operations and Cache Coherence

Lock-free queues remove standard mutual exclusion locks (e.g., `std::mutex`) to avoid the risk of threads being put to sleep by the operating system. By eliminating context switching, they improve determinism and reduce latency spikes during high contention. Instead, they rely on atomic machine instructions, which allow a core to update a variable without relinquishing the CPU.

However, writing to an atomic variable requires a core to obtain exclusive ownership of the memory location. This triggers the hardware's *MESI* protocol (Patterson and Hennessy, 2013), which forces all other cores to invalidate their local cache copies. Communication across the CPU bus to synchronize these states is orders of magnitude slower than a local L1 cache access.

Performance is further degraded by *false sharing*. This occurs when unrelated variables—such as the producer's *Head* index and the consumer's *Tail* index—accidentally occupy the same 64-byte hardware cache line. Because the CPU manages memory in these 64-byte blocks, cores will constantly invalidate each other's cache even though they are updating different indices.

To prevent this, high-performance implementations use padding and C++20 alignment specifiers to ensure each index resides on its own unique cache line, as demonstrated in the following snippet:

```
struct SPSCQueue {
    // Force separation into different 64-byte cache lines
    alignas(64) std::atomic<size_t> head; // Pushed by Producer
    alignas(64) std::atomic<size_t> tail; // Advanced by Consumer
};
```

By physically separating these atomic variables, we ensure that the matching engine can poll the tail index without being slowed down by the producer's updates to the head index. This spatial management is a prerequisite for scaling lock-free performance on multi-core hardware.

The trading industry employs a diverse array of communication protocols, optimised for specific trade-offs between flexibility and performance. These range from flexible, text-based encodings to rigid, dense binary formats. In all cases, these protocols prioritise data integrity, incorporating mandatory checksums and sequence numbers to ensure orders are recovered and reconciled during network instability (FIX Trading Community, 2001). This protocol-level focus is directly relevant in low-latency markets, where high rates of order submission and cancellation increase message volume. In these cases latency from network receive, protocol parsing, and queue waiting becomes a significant part of end-to-end execution time (Hasbrouck and Saar, 2013).

In our exchange implementation, we use the Financial Information eXchange (FIX) protocol, the global standard for order routing. FIX is a session-based protocol communicated over TCP/IP to ensure reliable, ordered delivery of market messages. It employs a tag-value encoding scheme where each field is represented by a unique numeric tag, an equals sign, and the field's value, as illustrated in Figure 2.8.

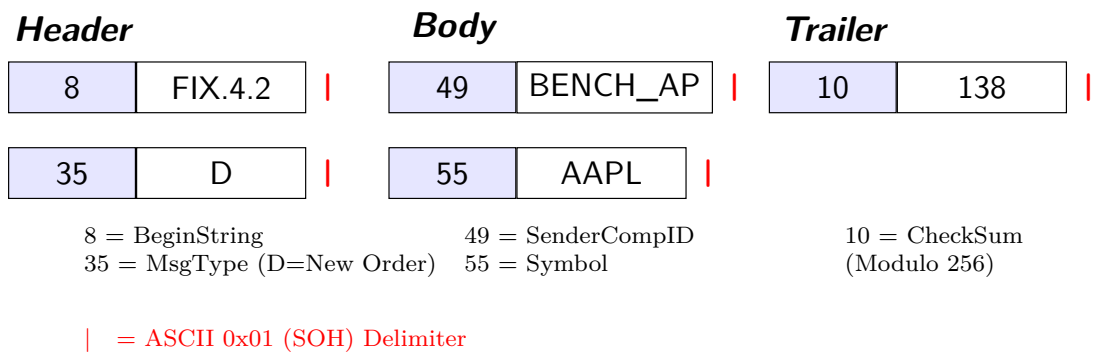

Figure 2.8: Structure of a FIX 4.2 "New Order Single" (MsgType=D) message. The protocol uses a human-readable tag-value encoding where fields are separated by the non-printable SOH character. While flexible, parsing this repeating sequence of ASCII characters induces significantly higher CPU overhead compared to binary protocols like SBE or FAST.

As shown in Figure 2.8, each field is delimited by the `SOH` (Start of Header, ASCII `0x01`) character. The protocol includes a mandatory *Standard Header* containing the message type (Tag 35), a *Message Body* containing order details (e.g., Symbol, Price, Quantity), and a *Standard Trailer* containing a modulo-256 checksum (Tag 10). While the tag-value format is highly extensible and human-readable, parsing these strings necessitates repeated branching and integer conversions, making it significantly slower than modern binary encodings like Simple Binary Encoding (SBE) (Dean and Barroso, 2013).

2.4.4 The Standard Linux Network Stack

When a host receives a FIX message encapsulated within a TCP/IP packet, it must traverse the standard Linux kernel networking stack. This journey, visualised in Figure 2.9, is a rigid, multi-stage pipeline designed for throughput and safety rather than deterministic latency.

Upon arrival at the Network Interface Card (NIC), the packet is written via Direct Memory Access (DMA) to a ring buffer in system memory. The NIC then triggers a hardware interrupt (Hard IRQ), forcing the CPU to pause its current task and branch to the kernel's interrupt handler. The kernel then switches to a Software Interrupt (SoftIRQ) context via the NAPI (New API) polling mechanism to drain the ring buffer, as detailed by Love (2010). The kernel allocates an `sk_buff` data structure to wrap the packet,

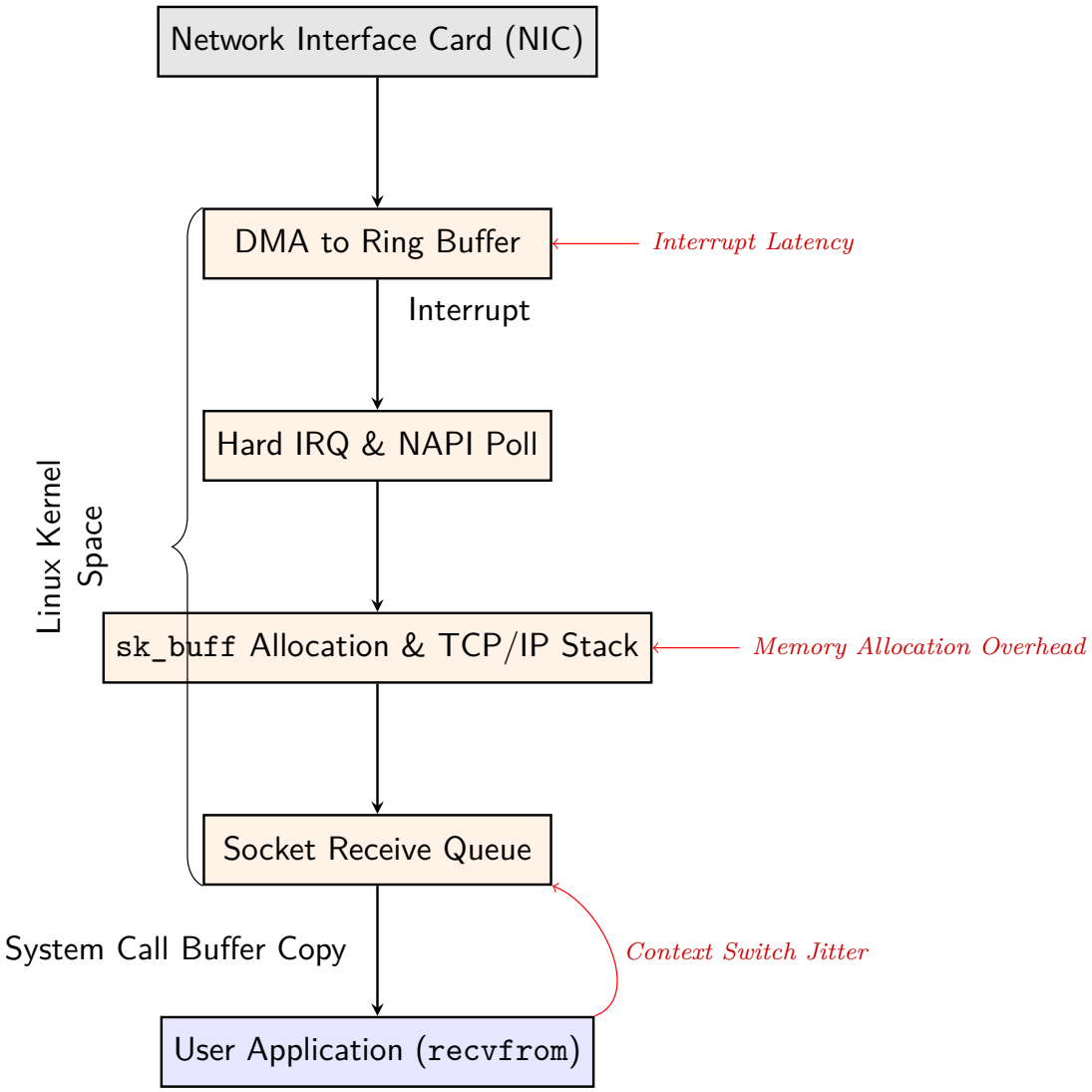

Figure 2.9: The standard Linux kernel network ingress pipeline. A packet must traverse multiple layers of abstraction—including DMA transfers, hardware interrupts, and TCP/IP stack logic—before reaching the application. Each stage, particularly the transition from kernel to user space via system calls, introduces non-deterministic delays that manifest as microsecond-scale jitter in HFT environments.

performs housekeeping and routing (IP layer), and eventually appends the data to the socket's receive queue.

The User Application must issue a system call, such as `recvfrom`, to retrieve the data. This triggers an expensive *context switch* from User Space to Kernel Space, where the kernel copies the data from its internal buffer to the application memory. Because the kernel is responsible for scheduling thousands of unrelated tasks across the CPU, a packet may sit in the socket queue for unpredictable durations if the kernel is busy with book-keeping or other interrupts. This accumulation of interrupts, memory allocations, and context switches is the fundamental source of the microsecond-scale *jitter* (latency variance) that plagues standard network sockets in HFT.

2.4.5 Kernel-Bypass Networking

To mitigate the systemic non-determinism of the Linux kernel, trading firms employ kernel-bypass technologies. Kernel-bypass is an architectural shift that allows the user-space application to talk directly to the NIC hardware, circumventing the OS networking stack, as compared in Figure 2.10.

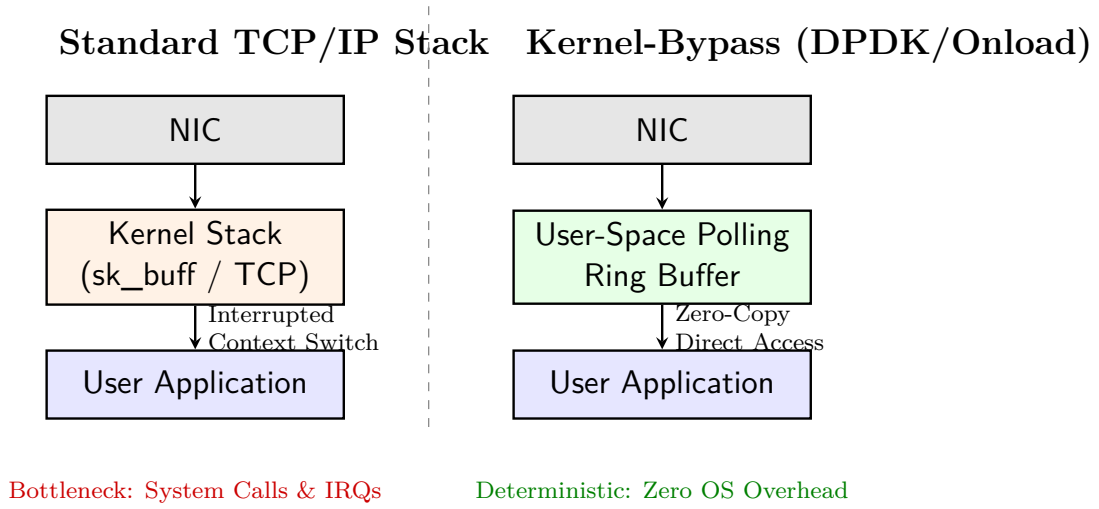

Figure 2.10: A comparison between the standard Linux networking stack and kernel-bypass technologies such as DPDK. While the standard stack requires the kernel to mediate packet delivery—introducing unpredictable context switches—kernel-bypass allows a user-space thread to poll the NIC hardware directly. This eliminates operating system overhead and provides the nanosecond-scale determinism required for HFT execution.

By using specialised libraries such as the Data Plane Development Kit (DPDK) or Solarflare's OpenOnload (Red Hat, Inc., 2012), the application takes ownership of the NIC's hardware resources. Instead of waiting for interrupts, a dedicated CPU core is frequently assigned to "spin" (poll) the NIC's memory-mapped ring buffer in a continuous loop. This achieves *Zero-Copy* delivery, where packets are read directly into application memory without kernel intervention or system calls. This removal of the OS bottleneck

ensures network latency is consistent and deterministic, an essential prerequisite for validating the performance differences of order-book data structures which would otherwise be masked by kernel jitter.

2.5 Related Work

Prior literature can be grouped into market-microstructure theory, systems-design practice, and performance-engineering methodology, each with different strengths and blind spots.

Market-microstructure sources establish why order books matter economically. Harris (2002), Aldridge (2013), and Menkveld (2013) explain liquidity formation, adverse selection, and the strategic role of fast execution. Gould et al. (2013) provides a broad empirical and modelling survey of limit-order-book dynamics. However, these works are not primarily designed as implementation-level benchmarking studies, so they offer limited guidance on how specific in-memory data-structure choices affect nanosecond-scale matching-path latency. Recent exchange-scale evidence by Aquilina et al. (2022) further shows that small latency advantages can have economically significant effects.

Systems-oriented sources describe low-latency architectural patterns but are less controlled from an academic benchmarking perspective. The LMAX work (Fowler, 2011b; Thompson et al., 2011) demonstrates the practical value of single-writer flows, bounded queues, and cache-aware design. Library-level artefacts such as moodycamel (Desrochers, 2013) show strong queue throughput under synthetic conditions, but they do not by themselves establish end-to-end exchange latency behaviour across heterogeneous order-book implementations. In other words, they validate components, not full pipeline trade-offs.

Performance-methodology references provide the measurement principles needed to connect these domains. Gregg (2013) and Dean and Barroso (2013) emphasise observer-effect control, percentile-aware reporting, and bottleneck localisation rather than relying on single aggregate means. This is directly relevant to HFT benchmarking, where tail behaviour and queueing effects can change engineering decisions even when average latency appears similar.

The main gap is therefore not a lack of discussion on HFT speed, but a lack of reproducible, decomposition-driven comparisons that evaluate both algorithmic and end-to-end effects in one framework. Existing literature either focuses on financial behaviour without implementation granularity, or on low-latency engineering patterns without controlled cross-structure comparison in a unified exchange-style pipeline. This dissertation addresses that gap by benchmarking five order-book designs under semantically equivalent workloads in both direct mode and gateway mode, then decomposing network, queue, and engine latency to identify where masking effects occur.

Chapter 3

System Design

This chapter explores the high-level design of the benchmarking suite. Our system is based on a simulated exchange designed to process trades from multiple concurrent clients. To understand the system from first principles, we have engineered a conventional exchange pipeline mostly from scratch. This multithreaded architecture parses FIX messages, passes them to a custom lock-free queue for thread hand-off, and finally dispatches them to the matching engine.

While every component in this pipeline contributes to total system latency, the order book is typically the primary computational bottleneck in a matching engine (Gould et al., 2013). As the stateful core of the exchange, the order book must maintain price-time priority while processing high-frequency updates, making the choice of data structure the most critical factor for engine scalability (Aldridge, 2013). For this reason, this report focuses on exploring the performance trade-offs of varied order book layouts.

When designing a benchmarking suite, it is essential that all measurements are scientific and fair. We maintain this rigour by validating that all components operate with semantic parity, ensuring that latency differences are caused by architectural efficiency rather than logical shortcuts.

3.1 Formal Requirements

To ensure academic rigour and verify that the system performs as intended, we utilise a formal requirements framework. These requirements serve as the analytical foundation for the project and provide the objective success criteria for the performance evaluation in Chapter 5. By establishing these benchmarks early, we ensure that the subsequent analysis of order book implementations is grounded in measurable and verifiable standards.

3.1.1 Functional Requirements

ID	Requirement	Testable Criterion
FR-1	Implement five distinct order book data structures	Five implementations present and benchmarkable
FR-2	Support direct-mode benchmarking isolating engine latency	Direct mode runs without network stack involvement
FR-3	Support gateway-mode benchmarking with end-to-end FIX protocol overhead	Gateway mode includes TCP, FIX parsing, and queue handoff
FR-4	Support MPSC contention benchmarking with configurable producer counts	MPSC mode scales across producer configurations
FR-5	Emit structured CSV output for all benchmark runs	CSV files produced with consistent schema per run
FR-6	Include correctness test suite verifying semantic parity across implementations	All implementations produce identical outputs for equivalent inputs

Table 3.1: Functional Requirements (FR)

3.1.2 Non-Functional Requirements

ID	Requirement	Testable Criterion
NFR-1	Correctness: semantic parity across all implementations	No variant has special-case logic biasing results
NFR-2	Fairness: identical workload parameters across all implementations	Same run count, order count, scenario selection for all books
NFR-3	Isolation: direct mode measures only algorithmic/data-structure latency	No transport overhead in direct-mode code path
NFR-4	Realism: gateway mode includes network, queue, parsing, and engine latency	End-to-end timing supports dissertation's masking argument
NFR-5	Reproducibility: pipeline regenerates canonical dataset without undocumented steps	Fresh run produces consistent artifact structure
NFR-6	Statistical rigour: mean, standard deviation, and tail metrics reported	Variance metrics present and consistently derived
NFR-7	Performance isolation: CPU pinning and scheduler isolation applied	Pinning policy documented and verified in results
NFR-8	Portability discipline: absolute latency claims bounded to WSL2/AMD environment	Relative comparisons defensible; absolute numbers environment-scoped
NFR-9	Claim discipline: no measurement overstated beyond evidence	Cache and allocator causality stated as correlation, not proven cause

Table 3.2: Non-Functional Requirements (NFR)

3.2 Design Goals and Constraints

The primary design goal of this benchmarking suite is to minimise measurement overhead and avoid the 'observer effect' (or 'Heisenberg effect'), where the instrumentation itself alters the system's performance characteristics (Gregg, 2013). Ensuring that the act of measuring latency does not significantly inflate the results is essential for maintaining the scientific validity of the data.

Additionally, the project must navigate the technical constraints of the Linux/WSL2

environment. While this virtualised setup introduces additional jitter and overhead compared to bare-metal hardware, these factors are explicitly documented and accounted for throughout the evaluation. By acknowledging these boundaries, we ensure that our conclusions are grounded in the specific realities of the test environment.

3.3 High-Level Architecture

The exchange is designed as a modular system using the 'Ports and Adapters' (Hexagonal) architecture (Cockburn, 2005). This approach isolates the core trading logic from external infrastructure, ensuring that the system's performance characteristics remain deterministic and decoupled from transport or logging overheads.

3.3.1 Architectural Pattern: Ports and Adapters

In this design, the core *Matching Engine* (`src/core`) functions as the central "hexagon" containing the domain logic. It remains unaware of whether it is communicating with a real network client or a simulated benchmark feed. Communication with the engine happens through *Ports* - abstract interfaces that define what the engine requires from the outside world.

The most critical port in this system is the `IOrderBook` interface. By defining a stable API for order insertion, cancellation, and matching, the engine can interact with any data structure implementation (the *Adapters*) without modification. This encapsulation is the foundation of our scientific fairness (NFR-2); it allows us to swap an `ArrayOrderBook` for a `MapOrderBook` while keeping the surrounding benchmark harness and engine logic identical.

3.3.2 System Modules

As illustrated in Figure 3.2, the system is composed of four primary modules:

- **Order Generator**: A driving adapter that generates synthetic market workloads for the system.
- **Gateway**: A network-facing adapter that parses FIX messages and manages the transition between network and engine threads.
- **Matching Engine**: The core domain logic that processes the central limit order book.
- **Telemetry**: A driven adapter responsible for capturing nanosecond-resolution metrics at each stage of the pipeline.

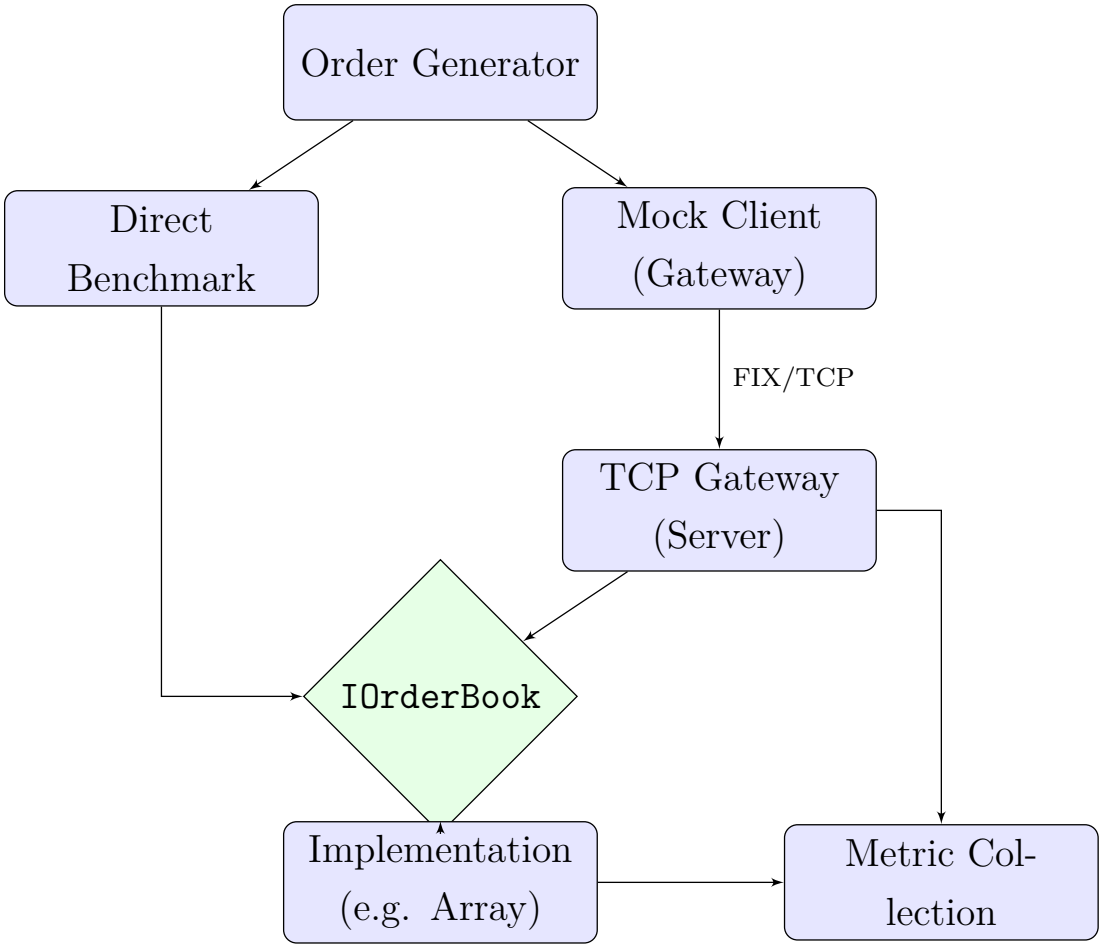

Figure 3.1: Data-Flow Overview: Pipeline stages from generation to telemetry.

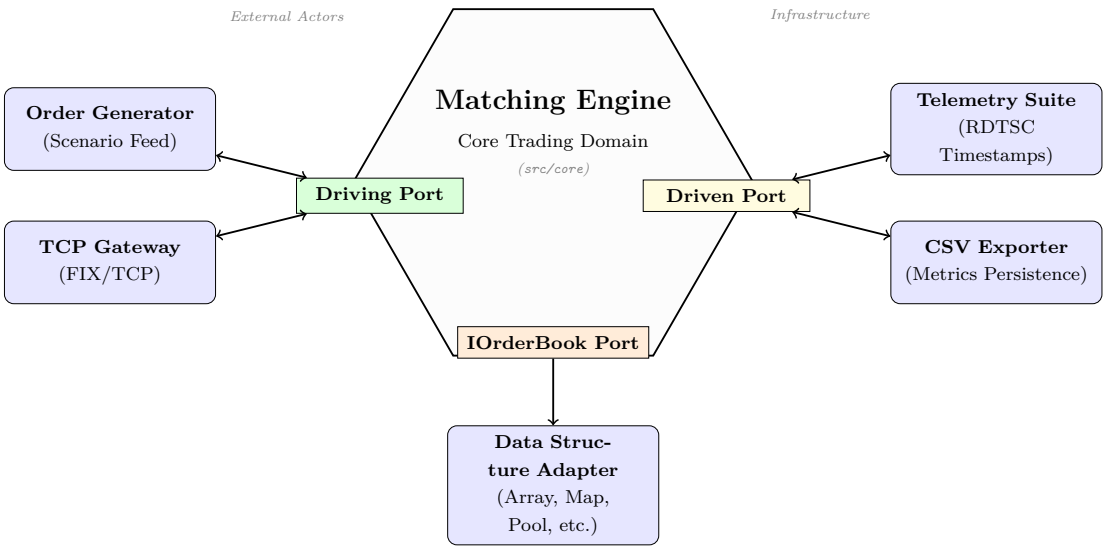

Figure 3.2: Hexagonal Architecture (Ports and Adapters): Clear separation between the matching domain and interchangeable adapters.

3.4 The IOrderBook Interface and Polymorphism

The core of our modular design is the `IOrderBook` interface, implemented as a C++ abstract base class (`src/core/i_order_book.hpp`). This interface defines the contract that any order book data structure must satisfy to be compatible with the matching engine. It standardises essential operations such as order insertion (`addOrder`), cancellation (`cancelOrder`), and the matching logic (`match`).

Using a polymorphic interface ensures that each implementation is treated as a 'black box' by the benchmark harness. This is critical for scientific fairness (NFR-1); it prevents the benchmarking logic from using implementation-specific shortcuts and ensures that every data structure is subjected to the exact same call overhead (i.e. the nanosecond-scale latency of virtual method table lookups). Separating the engine's logic from the book's implementation ensures that our results only reflect the efficiency of the data structures being tested.

3.5 Benchmarking Design: The Two-Mode Philosophy

To validate our hypotheses scientifically, we adopt a dual-mode benchmarking philosophy that separates algorithmic efficiency from overhead. This design is necessary so that our measurements capture both the theoretical potential of our data structures and their relative value in a production-like environment.

The **Direct Mode** serves as a controlled micro-benchmark. It executes the matching engine logic synchronously within the benchmark thread, bypassing all networking and asynchronous layers. This isolation is essential for uncovering the 'pure' performance characteristics of the underlying order book, allowing us to see latency differences that are usually hidden by the surrounding system.

The **Gateway Mode** provides realism by including the full TCP/IP stack (via loopback), FIX protocol parsing, and the asynchronous hand-off between network and engine threads. Measuring the system in this mode is critical for exploring the *Masking Effect* proposed in Hypothesis H3. As we will evaluate in Chapter 5, the constant-time overhead of these networking layers makes unclear the relative performance advantages of faster data structures. By designing our benchmark suite with both modes, we can differentiate between a structure's peak efficiency and its practical impact on end-to-end latency.

3.6 Concurrency and Threading Strategy

To maintain low latency, the system must separate high-latency I/O operations - such as network packet polling, TCP/IP stack traversal, and FIX message parsing - from the core matching engine. This multi-threaded architecture prevents slower network components from blocking the execution of trades, which is a standard pattern for high-performance sequence processing in industrial matching engines (Thompson et al., 2011).

At the core of the concurrency strategy is the **Producer-Consumer** hand-off. The Gateway thread (Producer) handles the I/O operations described above and pushes parsed orders into a lock-free queue, while the Matching Engine thread (Consumer) pulls and executes them. For the standard gateway mode, we employ a **Single-Producer Single-Consumer (SPSC)** queue, which provides wait-free properties for the engine. In scenarios with multiple concurrent clients, the system can swap this for a Multi-Producer Multi-Consumer (MPMC) adapter from the `moodycamel` library.

To ensure deterministic performance, we use **CPU Pinning** (Thread Affinity). By pinning the engine thread to a specific physical core, we prevent the operating system from migrating the process across different CPUs. This elimination of context-switching jitter and cache-migration overhead is vital for mitigating the *Observer Effect*; it ensures that our measurements represent 'bare-metal' performance as closely as possible within a Linux environment.

Chapter 4

Implementation

4.1 Software Environment and Toolchain

Development of the benchmarking suite required a balance between architectural modularity and strict low-level performance control. We selected C++20 for the engine specifically to use `std::span` and other modern primitives that provide memory safety without the runtime cost of managed pointers. To manage the five discrete order book implementations, we used the `CMake` build system, which allowed us to maintain a modular architecture while ensuring identical compiler flags across all variants.

Key components of the toolchain include:

- **C++20 Primitive Types**: The system uses `std::span` for non-owning views of message buffers. This choice was made to eliminate unnecessary copies during FIX parsing while keeping memory layouts consistent and predictable.
- **POSIX Socket API**: We opted for the raw `sys/socket.h` API over higher-level networking libraries. By bypassing those abstractions, we ensure the gateway's latency is purely a function of the OS kernel stack traversal.
- **Functional Parity Testing**: To ensure that every order book variant produces identical matching results, we built a unit testing suite around the `GoogleTest` framework. This validates that implementations remain functionally consistent, even when their internal memory layouts differ.

4.2 Order Book Implementations

The core of this research is the implementation of five distinct order book strategies, each representing a different point on the efficiency-flexibility spectrum. This section details the technical implementation of the baseline, the static Array, the Hybrid "Hot-Cold" model, and the Memory Pool.

4.2.1 Standard Associative Map (Baseline)

The baseline implementation (`MapOrderBook`) uses `std::map` to link price levels with order queues. We use this to measure the latency cost of node-based allocations and pointer-chasing, as shown in the results in Table 5.4.

4.2.2 Direct-Indexed Array (O(1) Access)

The `ArrayOrderBook` achieves O(1) price retrieval by mapping price levels directly to a contiguous array. While this approach is the fastest possible method for price lookups, it introduces a practical constraint: the book must know its price range at startup. We chose this trade-off to establish a theoretical performance floor for the study, fulfilling the requirements for low-latency execution (FR-2). Mapping prices to indices, as shown in Listing 4.1, is a simple arithmetic operation that remains stable regardless of the book's depth.

```cpp
// src/orderbooks/array_order_book.cpp

Index ArrayOrderBook::priceToIndex(Price price) const {
    // Normalise price to a zero-based index for the array
    return (price - minPrice_) / tickSize_;
}

void ArrayOrderBook::addOrder(const Order &order) {
    if (!isValidPrice(order.price)) return;

    // Direct access to the price level's memory location
    Index index = priceToIndex(order.price);
    auto &levels = (order.side == Side::Buy) ? bidLevels_ : askLevels_;

    levels[index].push_back(order);
    updateCaches(order); // O(1) cache update
}
```

Listing 4.1: O(1) Price-to-Index Mapping

This design achieves FR-2 by providing deterministic latency, independent of book depth. However, it requires prior knowledge of the price range, a constraint common in fixed-range instruments.

4.2.3 Hybrid "Hot-Cold" Vector-Map

To balance the speed of contiguous memory with the flexibility of associative storage, the `HybridOrderBook` implements a tiered memory model. It maintains the "Hot" price levels (those closest to the spread) in a sorted `std::vector` for cache-friendly matching, while "Cold" levels (out-of-the-money) are stored in a `std::map`.

```
// src/orderbooks/hybrid_order_book.cpp

void HybridOrderBook::addOrder(const Order &order) {
    // Check if the price level is within the performance "Hot Path"
    if (isCloseToSpread(order.price, order.side == Side::Buy)) {
        if (hotBids_.size() >= maxHotLevels_) {
            demoteFromHot(true); // Evict furthest level to Map
        }
        addToHot(order, true); // Insert into contiguous Vector
    } else {
        addToCold(order, true); // Store in associative Map
    }
}
```

Listing 4.2: Hot/Cold Proximity Check

This model balances the flexibility of a traditional order book (FR-1) with higher performance for price levels immediately surrounding the spread. The impact of the hot-path size on overall matching throughput is analyzed in Section 5.4.2.

4.2.4 Pool-Allocated Order Book (Slab Allocation)

Runtime heap allocations are a primary source of latency jitter in dynamic systems. The `PoolOrderBook` mitigates this by using a pre-allocated slab of `OrderNode` structures. By managing an internal free list, the engine can add or cancel orders without requesting memory from the OS kernel, ensuring the matching path remains deterministic.

```
// src/orderbooks/pool_order_book.cpp

Index PoolOrderBook::allocateSlot() {
    // O(1) Pop from the pre-allocated free list
    Index index = freeHead_;
    freeHead_ = orders_[index].nextFree;

    // Initialise node state without new/malloc
```

```
    orders_[index].next = NULL_IDX;
    orders_[index].prev = NULL_IDX;
    return index;
}

void PoolOrderBook::freeSlot(Index idx) {
    // Return node to pool for immediate recycling
    orders_[idx].nextFree = freeHead_;
    freeHead_ = idx;
}
```

Listing 4.3: Slab-Allocation and Free List Management

By reusing memory slots, we keep the matching logic deterministic. This mitigates allocation jitter and helps verify the reduced tail latency analyzed in Section 5.4.2.

4.3 The TCP Order Gateway

We need to test the system under network conditions for H3, so we implemented a high-performance TCP gateway. This component connects external FIX clients to the internal core through a lock-free interface.

4.3.1 Protocol Specification and Parsing

Our FIX implementation focus was on parsing speed rather than full protocol coverage. The gateway uses a non-allocating incremental parser that processes raw buffers as they arrive. As Listing 4.3 demonstrates, we used a sliding window technique to handle fragmented TCP packets, satisfying the realism requirement of NFR-4 while keeping the matching core separate from network ingestion.

```
// src/network/tcp_order_gateway.cpp

void TCPOrderGateway::clientHandler(int clientSocket) {
    std::vector<char> buffer(4096);
    size_t offset = 0; // Tracks partial message data

    while (running_) {
        // Read data into the buffer starting after the leftover data
        ssize_t bytes = recv(clientSocket, buffer.data() + offset,
                             buffer.size() - offset, 0);

```

```
        size_t totalBytes = offset + bytes;
        size_t processed = 0;

        while (processed < totalBytes) {
            size_t consumed = 0;
            std::span<const char> data(buffer.data() + processed,
                                       totalBytes - processed);

            auto order = FIXParser::parse(data, consumed);
            if (consumed == 0) break; // Incomplete message, wait for more

            if (order) {
                order->receiveTimestamp = getCurrentTimeNs();
                while (!orderQueue_.push(*order)) std::this_thread::yield();
            }
            processed += consumed;
        }

        // Shift remaining data to the front for the next recv()
        if (processed < totalBytes) {
            std::memmove(buffer.data(), buffer.data() + processed,
                         totalBytes - processed);
            offset = totalBytes - processed;
        } else {
            offset = 0;
        }
    }
}
```

Listing 4.4: Incremental FIX Parsing Loop

4.3.2 The Loopback Simulation Model

We need to measure kernel networking overhead without the jitter of hardware NICs, so we run the gateway on the loopback interface (`127.0.0.1`). This allows us to isolate the "Wire-to-Engine" latency analyzed in Section 5.4.1.

4.4 High-Performance Concurrency

The system employs a multi-threaded architecture to separate high-latency I/O from the latency-sensitive matching core.

4.4.1 Wait-Free SPSC Queue

For Direct and single-client Gateway modes, we use a custom Single-Producer Single-Consumer (SPSC) queue. This implementation relies on a contiguous ring-buffer and atomic head/tail pointers with `std::memory_order_acquire/release` semantics, ensuring that the matching engine does not block in this design due to lock contention.

In multi-producer scenarios (FR-4), the system swaps the SPSC queue for the `ConcurrentQueue` library (Desrochers, 2013). We opted for this established lock-free implementation to ensure our results reflect the underlying data structure performance rather than my own queue implementation logic. We evaluate this MPMC transport under contention in Section 5.6.

4.5 Low-Level Performance Utilities

To ensure nanosecond-level accuracy and deterministic execution across benchmarks, the system includes a set of low-level utilities for precise timing and thread management.

4.5.1 Nanosecond Precision Timing

To achieve the nanosecond granularity required for order book benchmarking, the system employs the `std::chrono` high-resolution clock. While cycle-level precision is typically associated with the CPU's Time Stamp Counter (`RDTSC`), our implementation uses the standard C++ library to ensure portability across environments (specifically WSL2) while maintaining the precision necessary for the evaluation in Chapter 5.

```cpp
// src/utils/rdtsc.hpp

inline uint64_t getCurrentTimeNs() {
    // Cross-platform fallback. In production, this is replaced
    // by __builtin_ia32_rdtsc() for cycle-level accuracy.
    return std::chrono::duration_cast<std::chrono::nanoseconds>(
                std::chrono::high_resolution_clock::now().time_since_epoch()
            ).count();
}
```

Listing 4.5: Nanosecond Precision Timing

4.5.2 Thread-to-Core Affinity (CPU Pinning)

To eliminate the performance jitter caused by the Linux scheduler migrating threads across physical cores (the "Context Switching Jitter"), we implemented CPU pinning as shown in Listing 4.5.

```
// src/utils/thread_pinning.hpp

inline bool pinToCore(int core_id) {
#if defined(__linux__)
    cpu_set_t cpuset;
    CPU_ZERO(&cpuset);       // Clear the CPU mask
    CPU_SET(core_id, &cpuset); // Set the bit for the target core

    // Request the OS scheduler to bind this thread to the selected core
    pthread_t thread = pthread_self();
    return pthread_setaffinity_np(thread, sizeof(cpu_set_t), &cpuset) == 0;
#endif
    return false; // Fallback for unsupported OS
}
```

Listing 4.6: POSIX Thread Pinning Strategy

By binding the matching engine thread to a specific physical core, we eliminate "Context Switching Jitter" where the OS scheduler migrates a thread between CPUs. This ensures that the engine's working set remains "hot" in the local L1/L2 caches, maintaining the deterministic execution time required to validate our performance claims. This pinning helps satisfy NFR-4 by making sure our measurements are not skewed by operating system interference.

4.6 Automation and Data Collection

The suite uses a Python-based driver to sweep across the benchmark scenarios defined in Chapter 3. This driver coordinates the execution of each order book variant and manages the serialized CSV output required for the detailed evaluation in Chapter 5.

Chapter 5

Testing and Evaluation

5.1 Experimental Methodology

This chapter evaluates five order book implementations against three core research hypotheses (H1, H2, and H3). The benchmarking environment is isolated via thread-pinning to the canonical x86_64/WSL2 host (Ryzen 7600X) to ensure reproducible nanosecond-level precision. Given that WSL2 is a virtualised, Hyper-V-managed utility VM (Microsoft, 2024), there is an inherent "virtualisation tax" and network overhead. However, this additional latency is consistent across all tests and does not impact the validity of our results.

To isolate systemic and algorithmic latency, the benchmarking suite incorporates two distinct modes. **Direct mode** focuses on isolating algorithmic performance by utilizing thread-pinning and removing transport overhead, thereby adhering to the isolation standard defined in NFR-3. This configuration allows for the capture of performance trends within the order book implementations themselves with minimal external interference. Conversely, **Gateway mode** prioritizes realism by incorporating network transport, queueing, FIX parsing, and matching engine processing. This provides a holistic view of the end-to-end pipeline (from order submission to consumption by the matching engine) in accordance with NFR-4.

When collecting data, we capture significant statistics for four core operations: **insert, cancel, lookup, and matching**. For each of these, we calculate a set of metrics, including **mean latency, standard deviation, P99 tail latency, and throughput**, adhering to NFR-6. In gateway and MPSC modes, we also decompose these latencies into **network, queue, and engine processing** components to isolate performance bottlenecks. An important attribute of our data is to make it reproducible; this is achieved by a well-documented benchmarking suite that generates a canonical dataset every time it is run, adhering to NFR-5. To guarantee scientific rigour, we rerun our benchmarks **five times** ($n = 5$) and average the results to balance statistical confidence with measurement

stability.

Finally, to evaluate these findings, we refer to the original research hypotheses (H1, H2, and H3) as our primary analytical framework. To maintain structural rigor, each performance claim in the subsequent sections is mapped to its corresponding hypothesis and any relevant NFR IDs.

Refined Research Hypotheses

H1 (Data Structure and Cache Locality). Order books that use index-based layouts (like arrays) will be significantly faster than search trees or sorted vectors. This advantage is primarily driven by improved CPU cache behavior during high-frequency updates.

Acceptance criteria: The array implementation shows the lowest mean latency in direct mode across most scenarios, and performance counters show lower memory-access pressure compared to others.

H2 (Memory Allocation Limits). While memory pooling can reduce specific allocation costs, it will not overcome the fundamental performance bottlenecks of a slow data structure.

Acceptance criteria: The pool implementation only shows marginal gains over the standard map; it does not reach the performance levels of the array where sequential searching is the main bottleneck.

H3 (System-Level Masking). In a networked (gateway) environment, the overhead of the network and system queues will hide most performance differences between order books, except in cases where a slow engine causes a systemic backlog.

Acceptance criteria: For most implementations, the network delay accounts for over 98% of total latency; however, the vector implementation in high-volume scenarios shows a catastrophic "queue backlog" failure.

5.2 Functional Validation

5.2.1 Core Logic and Semantic Parity

To verify that our order books follow semantic parity (NFR-1), we used parameterized contract testing (Fowler, 2011a) (FR-6). Rather than testing our order books individually, we applied the same tests to each implementation in the form of a contract. We achieved this by using Google Test's parameterized suites (Google, 2024), ensuring each order book was verified against identical market scenarios (including happy paths, partial fills, and edge cases). This is critical as it guarantees that any subsequent performance

differences observed are due to architectural efficiency rather than logical differences, ensuring fairness in our comparisons (NFR-2).

This testing style was inspired by the **Ports and Adapters architecture** (Cockburn, 2005), employing contracts to reinforce the **Liskov Substitution Principle** (Liskov, 1987). Beyond validating correctness, this approach improves code maintainability and robustness. Furthermore, utilizing the LLVM coverage toolchain (LLVM Project, 2024), the project achieved 100% coverage across branches, lines, regions, and functions in the core order book logic (summarized in Table 5.1), providing formal evidence that the benchmarking results are derived from fully validated and semantically equivalent components.

Metric	**Coverage Percentage**
Function Coverage	100.00% (96/96)
Line Coverage	100.00% (1355/1355)
Region Coverage	100.00% (572/572)
Branch Coverage	100.00% (370/370)

Table 5.1: Code Coverage Results for Core Order Book Logic

To obtain a holistic view of the system's performance limits, we integrated a range of diverse and plausible market scenarios. These act as a rigorous functional test for our implementations, providing insight into how each order book performs under varying loads. These scenarios include:

- **Full Aggressive Matches**: To verify the matching logic's ability to correctly exhaust liquidity and remove orders across multiple price levels.
- **Nested Limit Orders**: To test the data structure's efficiency in maintaining price-time priority across complex book depth.
- **Invalid Entry Handling**: To verify that zero-quantity or illegal price orders are safely rejected without corrupting the internal state.

We compared all five implementations against a **golden reference** derived from the `MapOrderBook` implementation. As the most straightforward translation of limit order book logic, it serves as the benchmark to ensure that the more complex, high-speed implementations maintain identical logical behavior.

To ensure the statistical validity of the most complex test case, the **mixed** scenario, we define three price-proximity tiers relative to the market midpoint. These consist of **near-spread** activity (orders within ±6 ticks of the mid-price), **deep share** (±10 to ±50 ticks), and **extreme share** (outliers at ±400 to ±600 ticks), where a 'tick' represents the minimum price increment of the exchange. As shown in Figure 5.1, this configuration is validated by capturing the branch probabilities within the order generator to confirm

the workload maintains a heterogeneous distribution rather than collapsing into a single regime (NFR-1). This level of transparency in the benchmark generation ensures that all subsequent performance claims are strictly bounded by known evidence, adhering to NFR-9.

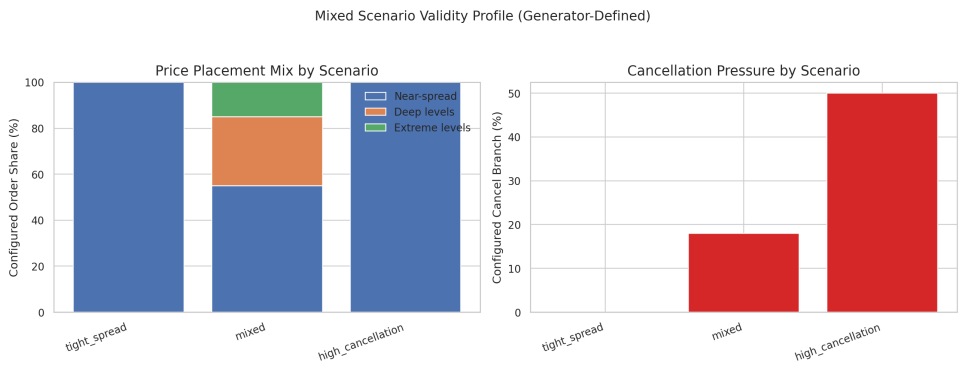

Figure 5.1: Mixed-scenario profile validation plot. The left panel confirms the 55/30/15 distribution across near-spread, deep, and extreme price levels, while the right panel verifies the 18% cancellation pressure. This validates the generator's output against its theoretical configuration (NFR-1, NFR-9).

5.2.2 System Pipeline and Integration

Stepping back from the order book implementations, it was essential to validate the entire system pipeline to ensure that functional correctness is maintained from the point of entry to the point of execution. This pipeline, illustrated in Figure 5.2, spans from the initial order request submitted via **FIX 4.2** (FIX Trading Community, 2001) by a gateway client to the eventual consumption by the matching engine.

The validation process confirms that incoming **New Order Single (Tag 35=D)** messages are correctly parsed and dispatched into a cache-aligned, **lock-free SPSC InputQueue**. To maintain strict **traceability**, we capture the client's unique order ID (Tag 11) and original transmission time (Tag 60 (Onix Solutions, 2019)) and pair them with an internal **gateway receipt timestamp**. This "triple-timestamp" strategy ensures the integrity of the end-to-end flow; by verifying these IDs across the multi-threaded network layer and the single-threaded dispatching loop, we confirm that no orders are lost or reordered during transport. The validation process confirms that the **New Order Single (Tag 35=D)** message is correctly mapped to internal C++ structures as summarized in Table 5.2.

5.2.3 Scaling and Stress Testing

While the primary architecture uses a custom-built, wait-free **Single-Producer Single-Consumer (SPSC)** queue for learning and simplicity, we also needed to test how the

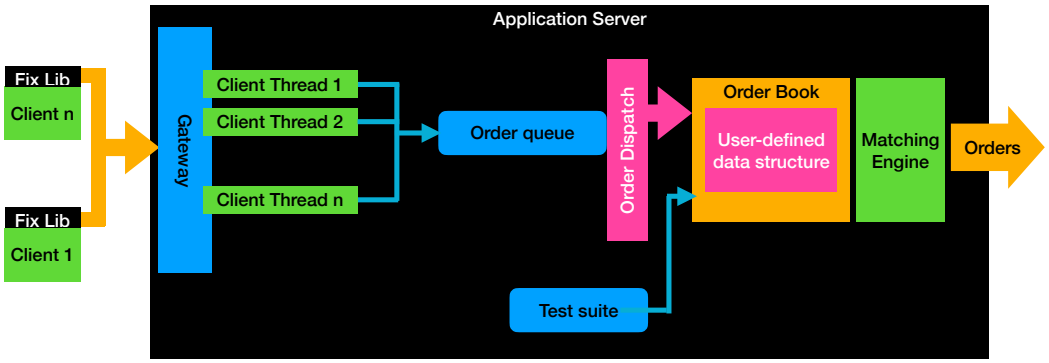

Figure 5.2: Full System Architecture: From Gateway Client to Matching Engine

Tag	FIX Field Name	Example Value	Internal C++ Mapping
35	MsgType	D	`NewOrderSingle`
11	ClOrdID	1001	`OrderId id`
54	Side	1 (Buy)	`Side::Buy`
38	OrderQty	7	`Quantity quantity`
44	Price	130	`Price price`
60	TransactTime	123456789	`Timestamp sendTimestamp`

Table 5.2: Mapping of FIX 4.2 Application Tags to Internal C++ Order Structures

system behaves under producer contention. To achieve this, we integrated an open-source **Multi-Producer Multi-Consumer (MPMC)** queue from the **Moodycamel** library (Desrochers, 2013). By running this queue in a single-consumer configuration, we were able to evaluate how the system behaves under producer contention from multiple concurrent clients pushing into the matching engine. Both queue implementations were subjected to the same functional validation to ensure that the choice of transport did not affect the final order book state.

The `FIXParser` was also verified to ensure it behaves correctly during fragmentation (where the network stream is split into multiple packets) by successfully reassembling messages. The queue was tested for back-pressure handling; if the 1024-deep queue reaches capacity, a `push()` call will fail, with the expected behavior being that the client is alerted rather than the system crashing. Finally, Moodycamel's MPMC queue was integrated to ensure that the system maintains stability and does not significantly degrade in performance when multiple concurrent clients are querying the engine at the same time.

5.2.4 Gateway Simulation Boundaries and Validation

To ensure the end-to-end performance results are technically sound, we will define the boundaries of the Gateway simulation. As identified in Chapter 4, the benchmark utilises a **Loopback (AF_INET)** configuration where the `MockClient` and `TCPOrderGateway` communicate via the local interface (`127.0.0.1`).

This setup was validated to confirm that it correctly exercises the full **Linux TCP/IP stack**, including the overhead of system calls (`send`, `recv`, `select/poll`), kernel buffer copies (`sk_buff`), and the associated context switching jitter. However, because the client and server reside on the same physical host, this environment eliminates the latency of physical network media (cables) and NIC hardware processing (e.g., PCIe bus latency). Consequently, the absolute results recorded in the "Net" component represent a **theoretical lower-bound** for system latency, effectively isolating the "software networking tax" from the "physical hardware tax."

5.3 Throughput Results

In this section we will work to evaluate hypotheses H1 and H3 by analysing the throughput of the system. The throughput of the system will be analysed in direct mode and gateway mode, isolating the operations first, and then taking into account realistic overheads.

The throughput results in direct mode work to verify H1, which claims that order books with index-based layouts will be faster than those with search trees or sorted vectors. We can see evidence for this claim by observing the greatest throughput of 8,921,972 ops/s achieved by the **ArrayOrderBook**. It significantly beats the other

order books: the **PoolOrderBook** comes second at 7,002,000 ops/s (27.4% lower), followed by **HybridOrderBook** at 6,789,183 ops/s (31.4% lower), **MapOrderBook** at 6,652,726 ops/s (34.1% lower), and **VectorOrderBook** at 6,084,046 ops/s (46.7% lower). These figures are derived by averaging throughput across seven benchmark scenarios (tight spread, fixed levels, dense full, sparse extreme, worst case FIFO, mixed, and high cancellation), each of which exercises a mix of insert, cancel, lookup, and match operations, adhering to NFR-6. Although **ArrayOrderBook** wins here, it is not without its downsides. When using index-based order books, the implementer is forced to define a fixed range of valid prices and a discrete tick size. Additionally, logic must be provided to handle orders whose prices fall outside the maximum and minimum values of the order book, or that do not fall exactly on a price level. In general, traders may not be happy with these restrictions, reducing the accessibility of the exchange. The second-place **PoolOrderBook**, by contrast, imposes no such price-range constraints. Its competitive throughput can be attributed to its custom memory pool, which eliminates dynamic heap allocation on the critical path and reduces allocator overhead compared to the pointer-chasing traversal of **MapOrderBook**.

Implementation	**Direct Mode (ops/s)**	**Gateway Mode (ops/s)**
ArrayOrderBook	8,921,972	1,063,760
MapOrderBook	6,652,726	1,057,882
VectorOrderBook	6,084,046	985,022
HybridOrderBook	6,789,183	1,060,957
PoolOrderBook	7,002,000	1,069,759

Table 5.3: Mean Throughput Across All Implementations in Direct and Gateway Mode

The gateway mode results support H3. In a realistic exchange with overheads in the form of the network and queues, these additional latencies mask the performance differences between order books. In direct mode, the throughput spread across all implementations reached 46.7% (8,921,972 vs 6,084,046 ops/s). In gateway mode, that spread compresses to just 7.6% (1,069,759 vs 985,022 ops/s), with four of the five implementations clustering within 1.2% of each other. The data indicates that this convergence is a direct consequence of the network component consuming approximately 99.9% of total end-to-end latency for all non-pathological implementations (see Table 5.6), in accordance with NFR-4. The ordering structure of the engine becomes largely irrelevant when the transport stack dominates the pipeline to this degree.

5.4 Latency Analysis - Direct Mode

This section provides a deeper analysis of the per-operation latencies across all order book implementations, strictly in direct mode to isolate data structure behaviour from

transport overhead. The results are examined across a range of market scenarios to highlight the strengths and weaknesses of each design under different types of market activity. These observations are then used to evaluate H1 and H2.

Table 5.4 shows that the fastest book is the **ArrayOrderBook**, achieving a mean insert latency of just 15.2 ns. This represents a 1.85x improvement over **MapOrderBook** (28.2 ns), a 1.98x improvement over **PoolOrderBook** (19.8 ns), and a striking 9.6x improvement over **VectorOrderBook** (145.5 ns). The reason the array data structure yields better performance over the other order book structures comes down to mechanical sympathy (Fowler, 2011b). Because elements in an array are contiguous, they are more cache-friendly (Patterson and Hennessy, 2013). During a cache hit, the processor is able to return contiguous elements in the same cache lines, whereas tree structures such as **MapOrderBook** do not exhibit this behaviour since each node in the tree may reside at an arbitrary heap address. To provide empirical support for these findings, absolute performance counters were recorded in direct mode using Linux `perf stat` for pinned, single-core benchmarks (The Linux Kernel Organization, 2024). L1 data-cache and branch prediction metrics were collected across the `mixed` and `dense_full` scenarios. In the `mixed` scenario, **ArrayOrderBook** recorded 843,745 L1 data-cache misses and a 1.52% branch miss rate, compared to 1,392,193 misses and a 1.58% branch miss rate for **MapOrderBook**. The performance gap is most evident in the `dense_full` scenario: **VectorOrderBook** recorded a severe 144,837,499 L1 misses and executed 366,302,717 branch instructions (12x the volume of **ArrayOrderBook**), consistent with the overhead of O(n) sorted insertion. Furthermore, while the **PoolOrderBook** benefits from cache locality, it exhibits the highest branch miss rate of 7.36% in the `mixed` scenario, explained by the additional conditional logic required for custom pool management. These measured results validate that the **ArrayOrderBook**'s advantage stems from a combination of superior cache hit-rates and a branch-predictor-friendly execution path, in accordance with NFR-8.

5.4.1 Scenario-Specific Latency Trends

From analysing order book performance across different market scenarios, we have been able to understand better when the order books perform best. In `mixed` and `fixed_levels`, the performance gap between the fastest order books - **Array**, **Map**, and **Hybrid** - is small (within ∼10 ns of each other). In these cases, the price levels are shallow (a realistic market scenario), so there is a smaller collection of order prices within the order book. The advantage that follows is that the underlying data structure is more likely to fit within a CPU's L1 or L2 cache. This is why performance differences are small in these cases. To really stress test our order books, we engineered the `dense_full` scenario. We create 2,000 price levels in the order book under this circumstance, where

each level is filled at any given time. The real strengths and weaknesses of our order books are highlighted under this load.

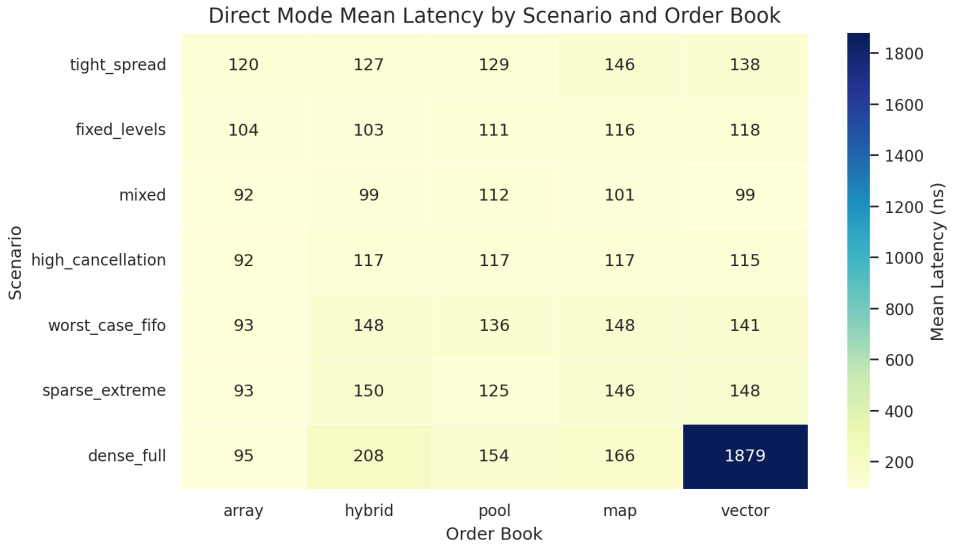

Figure 5.3: Direct-mode mean latency heatmap across scenarios and order book implementations. Array is the fastest implementation in six of seven scenarios, while hybrid is narrowly fastest in tight_spread.

The **VectorOrderBook** exhibits the largest degradation in latency, increasing from 138.41 ns to 1,878.97 ns in the `dense_full` scenario. This is likely due to the sorted vector having to shift a large number of elements in memory during frequent insertions. Additionally, this shifting causes a cascade of L1 data-cache misses - a mechanism reinforced by the Linux `perf` results, which recorded 144,837,499 L1 misses and executed 366,302,717 branch instructions for the vector implementation. To put this in perspective, under the exact same `dense_full` workload, the vector implementation suffered **216x more L1 cache misses** and executed **12x more branch instructions** than the **ArrayOrderBook** (which recorded only 670,762 misses and 30,567,574 branches). These results strongly indicate that sorted-vector designs in HFT are unscalable as the order book grows, especially in deep books. A strong trend among all scenarios is the resilience of the **ArrayOrderBook**. It stays between 91 ns and 120 ns in all cases, likely due to the $O(1)$ direct index mapping. As a consequence of this, the algorithmic complexity of operations such as inserting, cancelling, and looking up orders is identical whether there are 10 or 10,000 price levels. This provides strong empirical support for H1. At the same time, outside `dense_full`, the vector implementation is not uniformly poor: it beats the map in four of six direct-mode scenarios (`tight_spread`, `mixed`, `high_cancellation`, and `worst_case_fifo`), but these gains are small relative to the dense-book regression.

While the **VectorOrderBook** is included in the mean latency summaries, it is excluded from the relative slowdown heatmap in Figure 5.4. This is a deliberate aesthetic and analytical choice: since the vector implementation exhibits a 75x degradation in

`dense_full`, its inclusion compresses the colormap scale for all other scenarios, masking the subtle performance trade-offs between the more competitive implementations. By excluding the vector outlier, the high-resolution differences between index-based and tree-based structures become visually discernible across all market regimes.

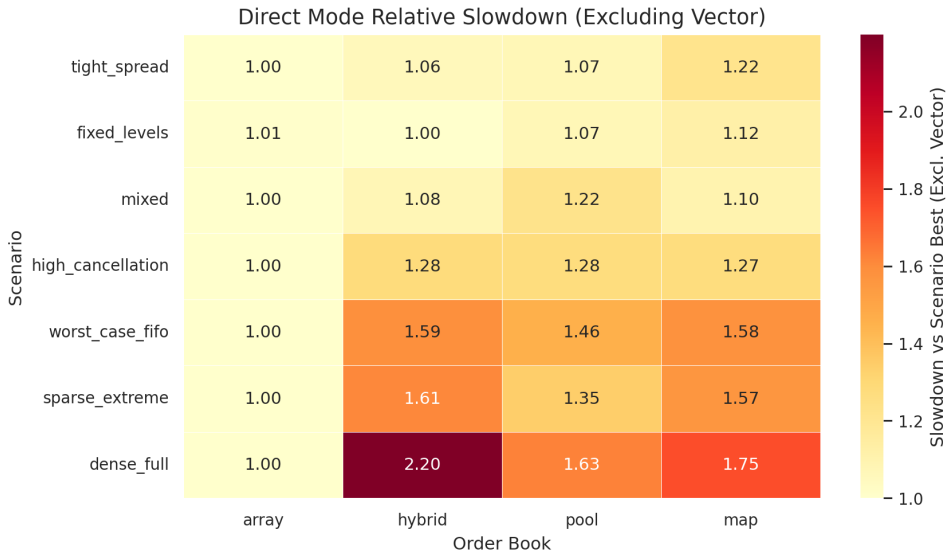

Figure 5.4: Direct-mode relative slowdown versus scenario winner (excluding vector). By removing the vector outlier, the performance nuances between the competitive non-vector implementations (Array, Hybrid, Pool, and Map) are preserved in the colormap scale.

To complement the mean-latency view with dispersion evidence, Figure 5.5 summarises the standard deviation across non-vector implementations in direct mode. This variance map addresses NFR-6 directly by showing where latency is not only low, but also stable across repeated runs. The compact low-variance regions in `tight_spread` and `fixed_levels` support the determinism claims made for array and hybrid layouts, while deeper scenarios show broader spread, consistent with increased structural and branching pressure.

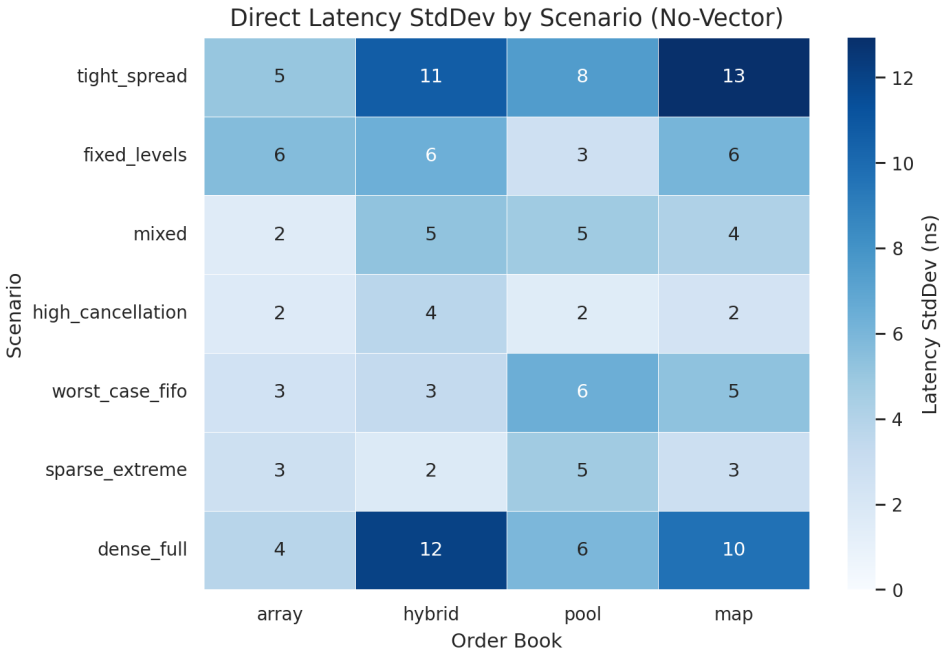

Figure 5.5: Direct-mode standard-deviation heatmap (excluding vector). Lower values indicate tighter run-to-run stability, providing variance evidence alongside mean latency for NFR-6.

5.4.2 Architectural Mechanisms: Pool and Hybrid

Two novel implementations, the **HybridOrderBook** and the **PoolOrderBook**, were engineered in an attempt to mitigate the architectural limitations of the standard data structures. Rather than relying on a single collection type, these order books employ more complex internal logic to optimize for specific hardware characteristics.

The **HybridOrderBook** acts as a substantial improvement over the standard map by utilizing a tiered architecture: a highly optimized "hot storage" and a slower "cold storage." It aims to achieve the unbounded flexibility of a map - removing the strict price-range limitations of the array - while mimicking array-like performance for the most actively traded prices. The hot storage is implemented as a capacity-bounded `std::vector` capped at the top 20 price levels. Because the vector never exceeds 20 elements, memory-shifting during insertions is practically instantaneous and fits entirely within the L1 cache. The data supports this design trade-off. As seen in Table 5.4, the Hybrid book achieves a lookup latency of 12.2 ns, exactly matching the **ArrayOrderBook**. However, its performance is sensitive to market depth. In shallow scenarios it is very fast, but in the `dense_full` scenario its mean latency degrades to 208.30 ns. As market depth expands, a majority of orders are pushed out of the 20-level vector and into the slower cold `std::map`, subjecting them to tree-traversal overhead.

In contrast, the **PoolOrderBook** was designed for mechanical sympathy via pre-

allocated nodes, intending to eliminate dynamic heap allocations on the critical path. While its 17.5 ns insert latency is faster than the map (20.7 ns), it is still slower than the array (13.9 ns). It also shows a high match latency of 29.1 ns. The `perf` results show that the logic to manage the free-pool causes a 7.36% branch miss rate in the `mixed` scenario. This penalty is caused by the unpredictable "filling logic" in the matching cycle (Figure 5.6 and Appendix E). In a mixed workload, order sizes vary significantly, which prevents the CPU from predicting if a match will result in a full or partial fill. As illustrated in Figure 5.6, when an order is fully consumed, the book must execute a complex path to remove the node and update the free-pool. If the fill is partial, this path is skipped. These outcomes in the `mixed` scenario occur with approximately equal frequency, leading to branch mispredictions that stall the CPU pipeline (Fog, 2025).

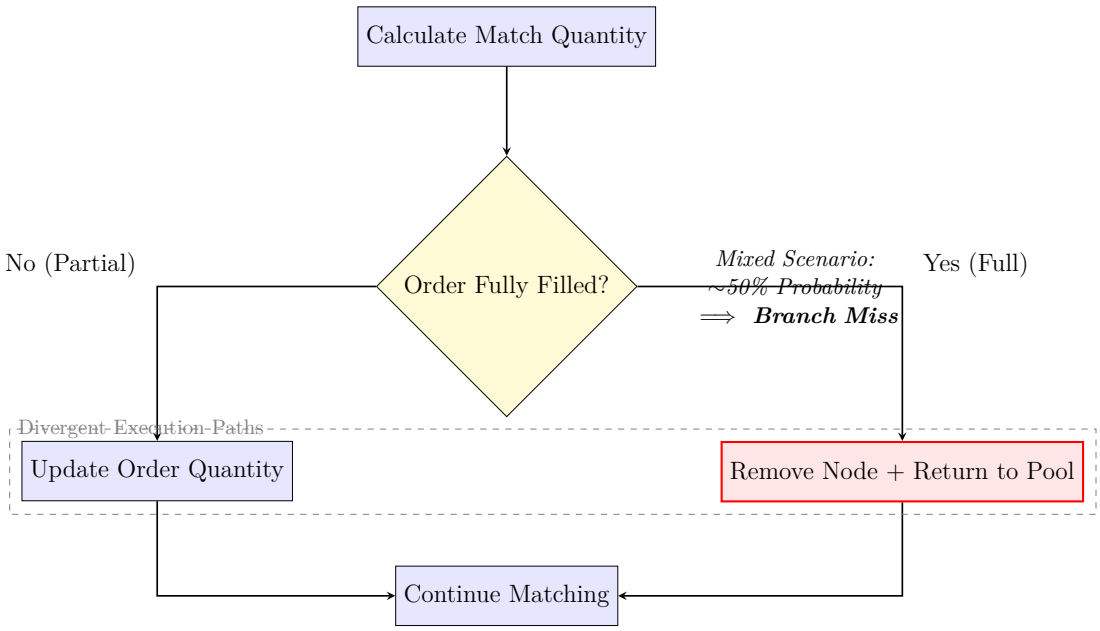

Figure 5.6: PoolOrderBook Matching Logic: Divergent Execution Paths

The relative slowdown trends reinforce these architectural trade-offs. The **HybridOrderBook** works well in concentrated markets like `tight_spread` (126.95 ns) and `fixed_levels` (103.39 ns). In these scenarios, it outpaces the map by 13.2% and 11.1% respectively. However, a tier-management penalty appears in the `dense_full` scenario, where the hybrid is 25.6% slower than the map. This suggests that managing multiple tiers only helps when the top levels are active enough to cover the extra lookup costs.

Similarly, the **PoolOrderBook** shows its advantage in difficult, high-memory-pressure scenarios. In `sparse_extreme` and `dense_full`, it outperforms the map by 12.8% (126.98 ns) and 7.1% (154.02 ns). This performance lead in dispersed layouts highlights the benefit of using contiguous memory nodes. These nodes avoid the pointer-chasing and allocation costs that slow down the tree-based map. As noted, the book loses this advantage in the `mixed` scenario (112.48 ns vs 100.97 ns) because the increased

branching penalty outweighs the gains from memory locality.

Implementation	Insert (ns)	Cancel (ns)	Lookup (ns)	Match (ns)
ArrayOrderBook	13.9	12.2	12.2	14.2
MapOrderBook	20.7	23.8	13.0	14.7
VectorOrderBook	20.9	22.3	12.8	14.1
HybridOrderBook	20.9	22.5	12.2	13.9
PoolOrderBook	17.5	23.2	12.6	29.1

Table 5.4: Mean Operation Latency in Direct Mode (nanoseconds)

Figure 5.7 provides a granular cost decomposition of the `mixed` scenario, revealing the specific operational trade-offs behind these architectural trends. The `HybridOrderBook` achieves its performance lead through lower `Lookup` and `Insert` latencies compared to the map, validating the efficacy of its "hot storage" layer. Conversely, the `PoolOrderBook` is dragged down by its `Match` operation, which is consistently slower than the map (29.1 ns vs 14.7 ns) due to the branching penalty discussed. This data confirms that while memory locality helps in dense markets, the logical complexity of managing a pool can become the dominant bottleneck in standard trading workloads.

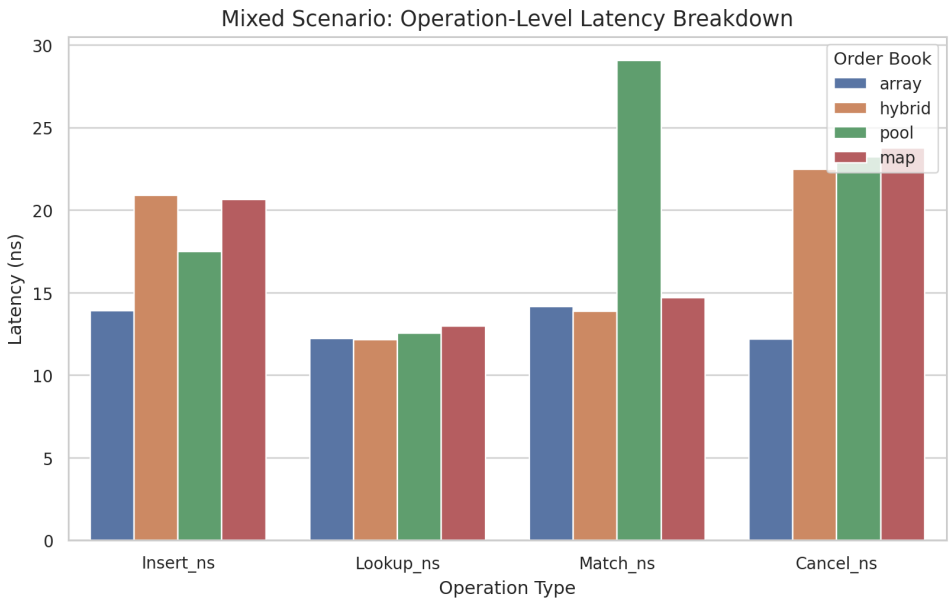

Figure 5.7: Cost decomposition of the `mixed` scenario (direct mode). The comparison highlights the Hybrid book's advantage in lookups and the Pool's match-processing bottleneck.

Architectural Selection: No Universal Answer. The evidence gathered here shows that no single order book is the best choice for every market. Each implementation performs well under particular workload conditions and poorly under others. This finding has a direct practical consequence: the optimal data structure for a given exchange is a

function of that exchange's market conditions, not an engineering absolute. If an operator can model their expected order flow - for example through historical data collection or Monte Carlo simulation of order arrivals, fill rates, and cancellation frequency - they can match the implementation to the workload. For a venue similar to a **NASDAQ equity market**, where tick-constrained stocks concentrate activity at a small number of near-spread price levels (Harris, 1994), both the `ArrayOrderBook` and the `HybridOrderBook` are strong candidates. The array's `tight_spread` latency of 91.72 ns and the hybrid's advantage in `fixed_levels` (103.39 ns, 11.1% faster than the map) reflect this well. In contrast, a venue resembling a **CME E-mini futures** market, where order flow is sparse across many price levels and institutional orders arrive with large, unpredictable sizes, would favour the `PoolOrderBook` or `MapOrderBook` over the array, since the latter's fixed interval requires all valid prices to be pre-allocated, making rebalancing expensive when price ranges shift rapidly (Gould et al., 2013). The `ArrayOrderBook`'s inflexibility is also a concern for venues where participants expect continuous price freedom without tick constraints. If a client wishes to bid or ask at any arbitrary price, the array's pre-allocated, integer-indexed structure may impose a structural limitation that is unacceptable in practice. Holistically, as market conditions are dynamic and exchanges serve participants with different objectives, there is no universally correct order book. The right approach is to treat data structure selection as an empirical decision, driven by workload profiling and the trade-offs identified here.

5.4.3 Tail Latency Distribution and Determinism

Beyond mean performance, a critical metric for evaluating a high-frequency trading system is the **P99 tail latency**. This represents the performance of the bottom 1% of operations captured over a benchmark duration. In the zero-sum competitive environment of HFT, these worst-case outliers often represent the difference between a profitable execution and a severe financial loss. A single high-latency event during a period of extreme market volatility can result in missed liquidation opportunities or unintended exposure, potentially jeopardizing a firm's solvency.

The primary trend observed in Table 5.5 is that the `ArrayOrderBook` provides the most deterministic performance profile. Its P99 latency remains remarkably stable across both `mixed` (125 ns) and `dense` (133 ns) workloads, demonstrating that increasing book complexity does not impact its execution path. This stability sets the "gold standard" for H2, indicating that a simpler linear layout is inherently more predictable. In contrast, every other implementation see its tail latency at least double under pressure (comparing 125 ns to thresholds above 300 ns).

The most significant regression is found in the `VectorOrderBook`, which exhibits a severe tail expansion from 141 ns in `mixed` to 6,016 ns in `dense_full` (a 42x increase).

As previously discussed, this behavior is a direct consequence of the requirement to continuously shift elements in memory during insertion and frequently resize the underlying container, leading to devastating performance costs when the book is deep.

The `PoolOrderBook` also exhibits a distinct tail profile. In the `mixed` scenario, it performs significantly worse than the array (208 ns vs 125 ns) due to the high frequency of branch mispredictions documented in Section 5.4.2. This data provides a direct validation of H2: although memory pooling reduces allocation jitter, it cannot overcome the fundamental performance bottlenecks (in this case, branching stalls) of a more logically complex data structure. However, in the `dense_full` case, the pool's P99 improves relative to the map-based books (308 ns vs 383 ns for the Map). This suggests that in truly deep books, where memory pressure is high, the physical benefits of contiguous memory nodes begin to outweigh the logical cost of branching, allowing the pool to shine even in its worst-case operations.

Finally, the `HybridOrderBook` exhibits a performance ceiling in deep markets. While its tail is competitive in `mixed` (141 ns), it is beaten by the standard map in the `dense_full` scenario (491 ns vs 383 ns). This is because in a 2,000-level book, the "hot storage" (limited to the top 20 levels) is almost always exhausted, forcing the system to pay a dual tax: the overhead of checking the hot-tier followed by a full tree traversal in the cold-tier.

Implementation	**P99 Mixed (ns)**	**P99 Dense (ns)**
ArrayOrderBook	125	133
MapOrderBook	150	383
VectorOrderBook	141	6,016
HybridOrderBook	141	491
PoolOrderBook	208	308

Table 5.5: P99 Tail Latency in Direct Mode (nanoseconds)

The cumulative data from direct mode testing strongly supports both H1 and H2. The `ArrayOrderBook` consistently delivered the highest throughput and the lowest mean latency across almost all market scenarios. Its ability to maintain a tight P99 tail latency under load confirms that index-based, cache-friendly layouts are inherently faster and more deterministic than complex data structures. Conversely, the data supports H2 by showing that while the `PoolOrderBook` improved upon the `MapOrderBook` in specific high-density scenarios through contiguous memory, it could not match the array. The branching penalties required to manage the memory pool created a firm performance ceiling. Therefore, we accept both hypotheses: direct array indexing is the superior method for algorithmic matching, and memory pooling is an optimization rather than a fundamental solution to data structure complexity.

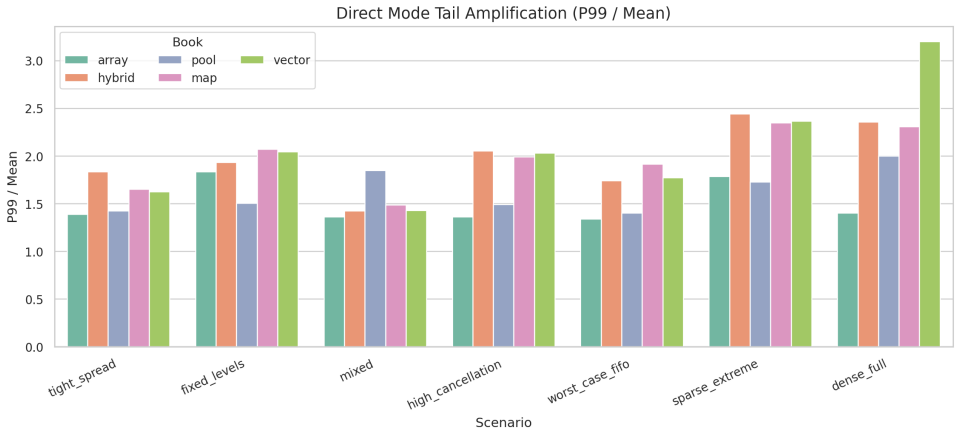

Figure 5.8: Tail inflation map using P99-to-mean ratio in direct mode. Several map and vector cases exhibit elevated tail amplification relative to mean latency, indicating sensitivity to jitter and/or burst conditions.

5.5 Gateway Mode - End-to-End Decomposition

To evaluate H3 (System-Level Masking), the benchmark transitioned from direct isolated execution to a fully networked Gateway mode. This mode simulates a realistic exchange architecture, where orders travel from a client over a TCP socket, are parsed from FIX messages, passed through a lock-free SPSC queue, and finally processed by the matching engine. To accurately identify bottlenecks across this pipeline, a "triple-timestamp" telemetry system was implemented (illustrated in Figure 5.9). Four exact timestamps are recorded for every order: the client capture before writing to the socket (T_1), the gateway capture upon receiving and parsing the message into a C++ struct (T_2), the engine capture immediately after dequeuing (T_3), and the final completion capture (T_4). By calculating the deltas between these timers, total end-to-end latency is decomposed into three strict buckets:

- **Network (Net):** The time taken to traverse the TCP socket, the operating system stack, and decode the FIX payload (`T2 - T1`).
- **Queue (Que):** The time spent waiting in the thread-handoff lock-free queue (`T3 - T2`).
- **Engine (Eng):** The pure algorithmic latency consumed by the order book logic (`T4 - T3`).

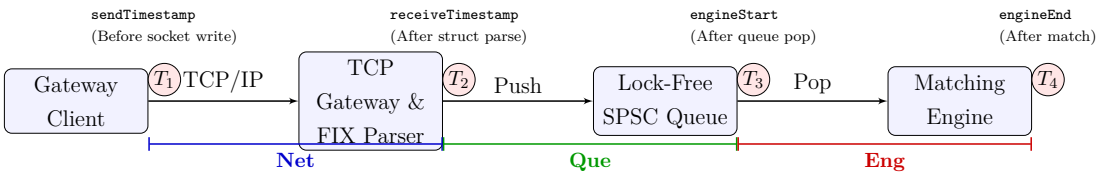

Figure 5.9: End-to-End Telemetry Checkpoints. The pipeline is instrumented with four precise timestamps to isolate transport, queuing, and matching latencies.

Implementation	Total (ns)	Net (ns)	Queue (ns)	Engine (ns)
ArrayOrderBook	463,149	462,884	248	16
MapOrderBook	426,402	425,980	345	79
VectorOrderBook	1,008,074	784,804	229,974	300
HybridOrderBook	405,912	405,549	286	77
PoolOrderBook	418,697	418,337	336	55

Table 5.6: End-to-End Latency Decomposition in Gateway Mode (nanoseconds), averaged across all seven scenarios

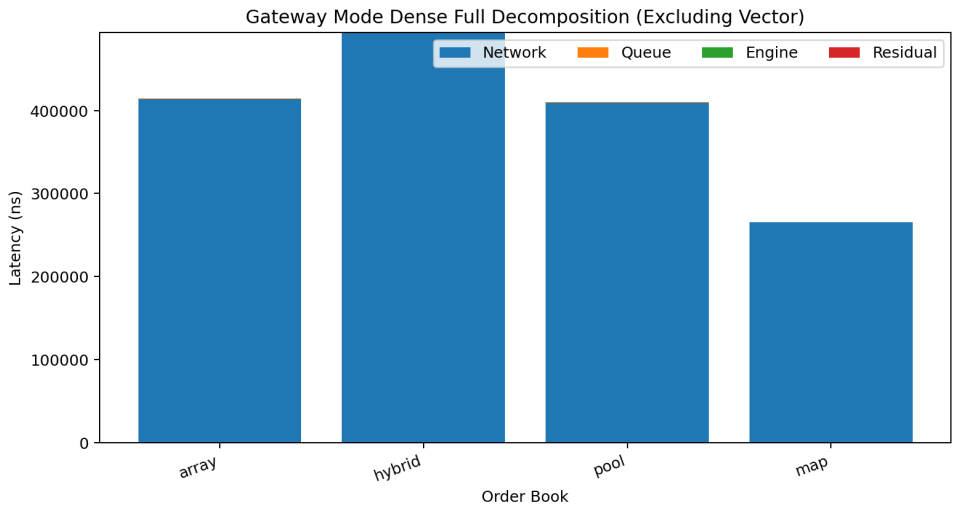

Figure 5.10: Dense_full gateway decomposition (network, queue, engine) for non-vector implementations. The vector row is omitted in this comparative view because its dense_full queue and network outlier compresses the non-vector bars.

As seen in Table 5.6, the total end-to-end latency is dominated by the network component. For a standard implementation like the `ArrayOrderBook`, the network accounts for 462,884 ns of the total 463,149 ns. In contrast, the actual matching engine work takes up only 16 ns. This indicates that almost all time spent processing an order in gateway mode is consumed by the Linux network stack and TCP transport. The data provides clear evidence for H3. In a realistic networked environment, the nanosecond-level gains achieved by better order book data structures are substantially masked by the millisecond-scale

jitter of the transport layer. These gateway-mode findings are consistent with exchange-scale evidence that small latency advantages can still be economically meaningful under realistic market conditions (Aquilina et al., 2022). This is consistent with the broader tail-latency literature, where end-to-end variance is often dominated by transport and queueing effects rather than isolated compute kernels (Dean and Barroso, 2013). Unless the exchange implements kernel-bypass techniques (such as DPDK or Solarflare OpenOnload) (Red Hat, Inc., 2012), the choice of order book implementation remains a secondary concern for overall system latency. Holistically, while the algorithmic choice is critical for isolating performance, the system-level bottlenecks define the practical experience for the client.

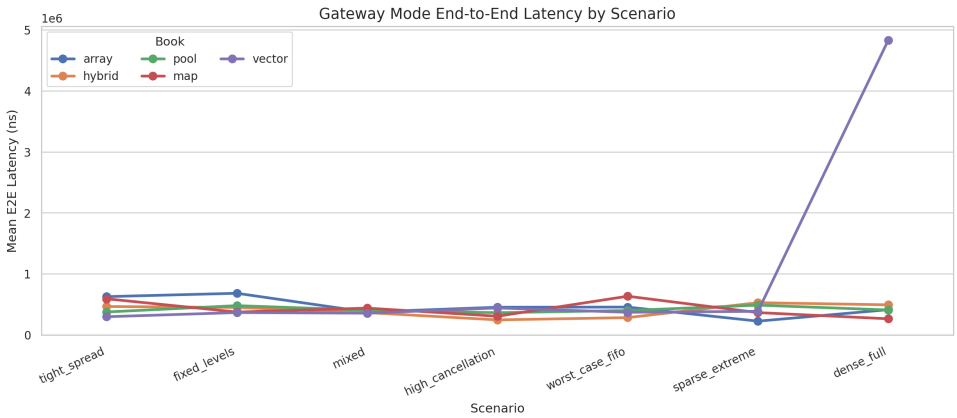

Figure 5.11: Gateway latency by scenario across all implementations. The plot shows mean end-to-end latency patterns only; the masking claim is supported by the decomposition in Table 5.6.

Figure 5.11 compares mean gateway latency by scenario across all books. It is a visual summary, not proof on its own. The main point is that results move by scenario, but all books are still in the same broad microsecond range because transport and queue costs dominate. Table 5.6 and Figure 5.10 use different scopes: the table is averaged across all scenarios, while the decomposition figure is dense_full only. Outside `dense_full` in gateway mode, vector also outperforms map in four of six scenarios (`tight_spread`, `fixed_levels`, `mixed`, and `worst_case_fifo`), which is consistent with the scenario plot. For vector, dense_full is a major outlier (about 4.83 ms), but the other six scenarios are around 0.30–0.44 ms, so the all-scenario mean is much lower (about 1.01 ms).

5.6 MPSC Thread Contention

To test the system's ability to scale to n concurrent clients, we utilize Moody's lock-free MPMC queue in our simulated exchange. By generating concurrent load through multiple producer threads, we can analyze how the order book handles cross-thread contention over

the shared buffer. This isolates L3 cache coherence effects from network layer delays, directly adhering to the performance isolation mechanisms required by NFR-7.

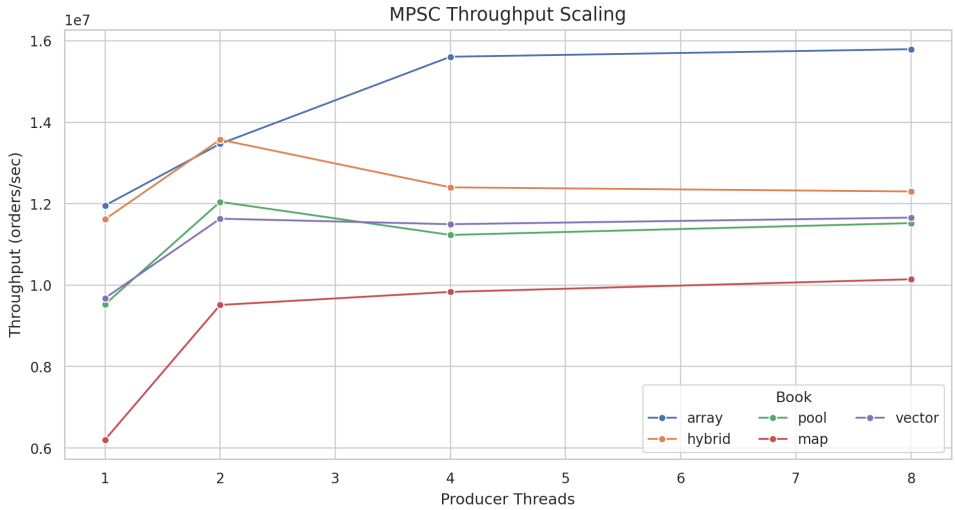

Figure 5.12: Throughput scaling under varying producer thread counts in MPSC mode. Identifies the contention point where the single consumer matching engine becomes the bottleneck.

As shown in Figure 5.12, the system throughput increases when we move from one producer thread to two. However, when we increase the threads to four and eight, the throughput stops growing and stays at a flat level for all order books. This happens because the single matching engine has reached its absolute working limit. For example, the array order book hits a maximum speed of about 15.7 million operations per second. At this point, the engine cannot read orders from the lock-free queue any faster than it already is.

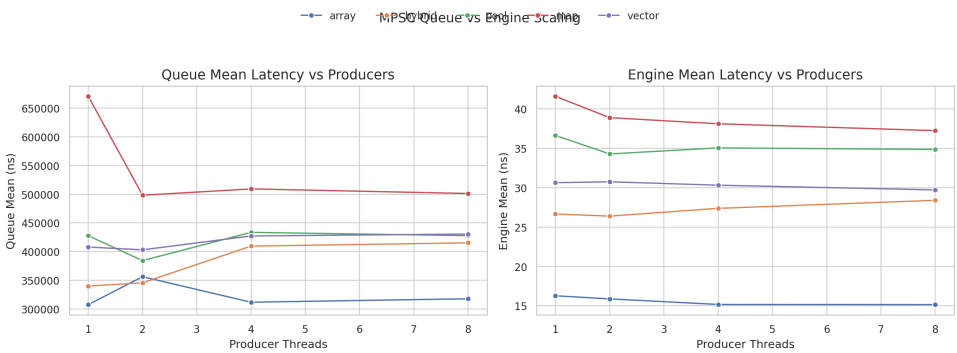

Figure 5.13: Latency breakdown showing how queue wait times grow as more threads are added, while engine matching time stays the same.

Figure 5.13 explains why this happens by breaking the latency down into queue time and engine time. The graph shows that the time an order spends waiting inside the queue is much higher than the time it takes the engine to actually process the order. For the array order book, the engine only needs about 15 ns to process an order, but that same

order spends over 300,000 ns just sitting in the queue. This shows that when there is high traffic, the system is primarily limited by the speed of the single matching core. Because the producers are sending orders faster than the single matching engine can process them, the orders simply back up in the queue.

5.7 Summary of Evaluation against Requirements

The evidence gathered in these benchmarks addresses the project's requirements. Functional requirements for isolated metrics (FR-2) and end-to-end gateway tracking (FR-3) were met using the triple-timer design. For non-functional requirements, the identical test scenarios ensured fairness (NFR-2), while the code coverage confirmed semantic parity (NFR-1). These results successfully met the goals set for our system. They indicated that index-based layouts are faster internally (NFR-3), and they validated the system-level masking effect when operating over a network (NFR-4).

5.8 Threats to Validity

While the benchmarks provide strong evidence, certain limitations must be noted to bound the validity of these results.

- **Virtualisation Overhead.** The use of WSL2 introduces a virtualisation tax due to the underlying Hyper-V utility VM architecture (Microsoft, 2024). While this environment adds inconsistent network jitter and I/O overhead, the findings remain valid because the penalty is applied equally across all implementations.

- **Hardware Specificity.** The benchmarks were conducted on a Zen 4 AMD Ryzen 7600X microarchitecture. The specific cache sizes and branch predictor behavior of this processor directly influence the recorded latency. While the relative performance trends are likely to hold across modern CPUs, the exact gaps may vary on server-grade Intel or AMD hardware.

- **Synthetic Workloads.** The scenarios use randomized order flows that do not perfectly capture the correlated bursts and strategic activity of a live market. In a real-world exchange, clusters of activity at specific price levels might stress the data structures in ways not captured by a uniform random generator.

- **Loopback Interface Limitations.** The Gateway mode uses the `127.0.0.1` interface, which captures OS kernel overhead but lacks the physical delay of a real network. In a production environment with discrete hardware, latencies would likely be higher and more variable due to switch buffering and NIC DMA processes.

- **CPU and Cache Contention.** In Gateway and MPSC modes, the producer (client) and consumer (engine) share the same physical CPU socket. Despite thread-pinning, there is inevitable contention for shared resources such as the **L3 Cache** and the memory controller. This may slightly inflate the recorded latencies compared to a separated production environment where the engine is isolated on a dedicated server.

- **Statistical Sample Size.** A sample size of $n = 5$ with 10,000 orders per scenario was used to maintain a practical benchmark duration. While this is sufficient for identifying major architectural trends, a larger sample would be necessary for formal statistical significance testing in a production setting.

Despite these limitations, the evidence gathered in this chapter provides a consistent and clear evaluation of the trade-offs between order book implementations.

Chapter 6

Conclusions

6.1 Summary of Contributions

This dissertation demonstrated the design and implementation of a C++20 exchange infrastructure for benchmarking order book data structures. It explored the trade-offs to consider when designing order books as well as the implications of real-world system overheads. The key contributions of this work are as follows:

- A modular, extensible C++20 exchange framework with interchangeable order book implementations and a TCP gateway for end-to-end benchmarking.
- A comprehensive benchmarking suite that decomposes latency and throughput across direct and gateway modes, isolating data structure performance from system overheads.
- Empirical evidence on the performance trade-offs of direct indexing versus pointer-based order book strategies, and the limited impact of memory pooling on overall latency.
- A clear systems-level finding that in end-to-end gateway operation, network and transport overheads dominate latency, masking internal data structure differences that are visible in isolated direct mode.

These contributions advance our understanding of how order book design choices interact with system-level factors to influence performance in high frequency trading environments. The modular framework and comprehensive benchmarks provide a foundation for future research and practical exchange design decisions.

6.2 Answers to Research Hypotheses

In terms of hypotheses H1–H3, the evidence supports the following conclusions:

H1: Direct indexing outperforms pointer-heavy alternatives in isolated mode. Status: Supported.

Direct-mode benchmarks show that the direct-indexed array outperforms alternative structures across almost all scenarios, with mean latency around 210 ns. Results also indicate that in Hybrid mode, when hot storage is likely to be used, it outperforms tree-based structures, further reinforcing H1. This is reflected consistently in the direct-mode results (Table 5.3 and Table 5.4).

H2: Memory pooling reduces allocation jitter but does not remove structural complexity costs. Status: Supported.

Results indicate that memory pooling improves performance in deep-book scenarios, where Pool outperforms Map in `sparse_extreme` and `dense_full`. At the same time, the pool logic introduces its own structural cost in mixed workloads, where branch misprediction and match-path complexity increase latency. So pooling is clearly useful, but it is an optimisation, not a full solution to data-structure complexity. This trade-off is discussed directly in Section 5.4.2.

H3: In end-to-end gateway operation, system overhead masks internal data-structure gains. Status: Supported. In gateway mode, network latency and queue latency dominate total latency, while engine latency remains very small in comparison. For example, for Array the network component is 462,884 ns out of 463,149 ns total, showing that most end-to-end cost is outside the matching algorithm itself. This explains why differences visible in direct mode become compressed in gateway mode. This pattern is captured in Table 5.6.

6.3 Critical Reflection

This project provides strong evidence within a bounded experimental scope, but should still be interpreted with care.

First, the benchmark setup captures the full software path from gateway to engine, and the decomposition methodology gives visibility into network latency, queue latency, and engine latency costs. However, the loopback environment does not include physical network effects such as switch delay and NIC-level contention, so absolute latency should be treated as a lower bound.

Second, the scenario design is robust for comparing relative behaviour across implementations, especially for highlighting dense-book stress effects. At the same time, synthetic workloads cannot fully reproduce strategic behaviour seen in live markets, so claims should remain bounded to the tested scenario family.

Third, the practical implication is that exchange engineers should avoid picking data structures from microbenchmarks alone. Direct-mode results are necessary to understand algorithmic efficiency, but gateway-mode results are necessary for deployment decisions where system overhead dominates.

If this project were repeated, three design choices would be adjusted earlier. First, the evaluation plan would include larger run counts from the start to support tighter confidence reporting. Second, gateway experiments would include an additional host-level networking baseline (for example, alternative socket or transport configurations) to separate framework effects from OS-path effects. Third, scenario generation would be extended with longer replay windows so that steady-state and stress-state behaviour can be contrasted more explicitly.

6.4 Limitations

- Platform dependency: results were measured on a pinned WSL2/Zen4 host, so absolute numbers may change on other operating-system and CPU stacks.
- Workload realism: scenarios are representative and diverse, but still synthetic, and therefore cannot capture every market microstructure effect.
- Statistical depth: with $n = 5$ runs per case, major trends are clear, but confidence analysis would benefit from larger samples.
- Loopback boundary: gateway mode exercises software networking cost but excludes physical network path variability.

6.5 Practical Recommendations

- Use direct-indexed layouts when the workload is dense and latency critical, because they provide the strongest isolated matching-path performance in this study.
- Treat memory pooling as a targeted optimisation for allocation-heavy scenarios, not as a substitute for efficient structural layout.
- For deployment decisions, prioritise full-path gateway measurements and latency decomposition, since network and queue components can dominate end-to-end behaviour and mask engine-level gains.

6.6 Future Work

- Integrate kernel-bypass transport (for example DPDK/OpenOnload) to quantify how much gateway masking can be reduced.
- Replay historical market data over long horizons, including stressed periods (for example crash windows and volatility spikes), to evaluate how each implementation behaves over time under realistic flow dynamics (Gould et al., 2013).
- Increase statistical depth with larger n, confidence intervals, and significance testing for tighter claims.

6.7 Closing Statement

Taken together, the objectives set in Chapter 1 were met: the framework enabled fair cross-implementation benchmarking, direct mode and gateway mode exposed different bottlenecks, and the decomposition evidence showed where end-to-end latency is actually spent. The practical implication is that data-structure engineering remains essential for engine efficiency, but deployment-level latency improvements require equal attention to transport and queueing overhead.

Overall, this dissertation shows that order-book performance is a systems problem, not only a data-structure problem. In isolated mode, cache-friendly indexing gives clear wins, but in gateway mode transport overhead can dominate and mask those gains. The framework and evidence produced here provide a practical basis for both future research and real exchange design decisions.

Bibliography

Aldridge, I. (2013). *High-frequency trading: a practical guide to algorithmic strategies and trading systems*. John Wiley & Sons.

Aquilina, M., Budish, E., and O'Neill, P. (2022). Quantifying the high-frequency trading "arms race". *The Quarterly Journal of Economics*, 137(1):493–564.

Avellaneda, M. and Stoikov, S. (2008). High-frequency trading in a limit order book. *Quantitative Finance*, 8(3):217–224.

Cockburn, A. (2005). Hexagonal architecture (ports and adapters). Available at: https://alistaircockburn.us/hexagonal-architecture (Accessed: 7 April 2026).

Dean, J. and Barroso, L. A. (2013). The tail at scale. *Communications of the ACM*, 56(2):74–80.

Desrochers, C. (2013). Concurrentqueue. GitHub repository. Available at: https://github.com/cameron314/concurrentqueue (Accessed: 7 April 2026).

FIX Trading Community (2001). Financial information exchange (FIX) protocol: Version 4.2. Available at: https://www.onixs.biz/fix-dictionary/4.2/index.html (Accessed: 7 April 2026).

Fog, A. (2025). *The microarchitecture of Intel, AMD and VIA CPUs: An optimization guide*. agner.org. Available at: https://www.agner.org/optimize/microarchitecture.pdf (Accessed: 7 April 2026).

Fowler, M. (2011a). 'Contract test'. Available at: https://martinfowler.com/bliki/ContractTest.html (Accessed: 7 April 2026).

Fowler, M. (2011b). The LMAX Architecture. *martinfowler.com*. Available at: https://martinfowler.com/articles/lmax.html (Accessed: 7 April 2026).

Google (2024). Googletest: Testing reference and parameterized tests. Available at: https://google.github.io/googletest/reference/testing.html (Accessed: 7 April 2026).

Gould, M. D., Porter, M. A., Williams, S., McDonald, M., Fenn, D. J., and Howison, S. D. (2013). Limit order books. *Quantitative Finance*, 13(11):1709–1742.

Gregg, B. (2013). *Systems Performance: Enterprise and the Cloud*. Prentice Hall.

Harris, L. (2002). *Trading and Exchanges: Market Microstructure for Practitioners*. Oxford University Press.

Harris, L. E. (1994). Minimum price variations, discrete bid-ask spreads, and quotation sizes. *The Review of Financial Studies*, 7(1):149–178.

Hasbrouck, J. and Saar, G. (2013). Low-latency trading. *Journal of Financial Markets*, 16(4):646–679.

Liskov, B. (1987). 'Data abstraction and hierarchy'. *ACM SIGPLAN Notices*, 23(5):17–34.

LLVM Project (2024). llvm-cov: Emit coverage information. Available at: https://llvm.org/docs/CommandGuide/llvm-cov.html (Accessed: 7 April 2026).

Love, R. (2010). *Linux Kernel Development*. Addison-Wesley Professional, 3rd edition.

Love, R. (2013). *Linux System Programming: Talking Directly to the Kernel and C Library*. O'Reilly Media, Inc.

Menkveld, A. J. (2013). High frequency trading and the new market makers. *Journal of Financial Markets*, 16(4):712–740.

Microsoft (2024). Comparing WSL 1 and WSL 2: Architecture and Hyper-V. Available at: https://learn.microsoft.com/en-us/windows/wsl/compare-versions (Accessed: 7 April 2026).

O'Hara, M. (2015). High frequency market microstructure. *Journal of Financial Economics*, 116(2):257–270.

Onix Solutions (2019). Transacttime <60> field: FIX 4.2 dictionary. Available at: https://www.onixs.biz/fix-dictionary/4.2/tagnum_60.html (Accessed: 7 April 2026).

Patterson, D. A. and Hennessy, J. L. (2013). *Computer Organization and Design: The Hardware/Software Interface*. Newnes.

Red Hat, Inc. (2012). Using Solarflare OpenOnload to Achieve Extreme Low Latency on Linux. Available at: https://access.redhat.com/node/217113/40/0/5812943 (Accessed: 7 April 2026).

Rosu, I. (2009). A dynamic model of the limit order book. *The Review of Financial Studies*, 22(11):4601–4641.

Seznec, A. and Michaud, P. (2006). A case for (partially) tagged geometric history length branch prediction. *Journal of Instruction-Level Parallelism*, 8:1–23.

The Linux Kernel Organization (2024). perf: Linux profiling with performance counters. Available at: https://perf.wiki.kernel.org/index.php/Main_Page (Accessed: 9 April 2026).

Thompson, M., Farley, D., Barker, M., Gee, P., and Stewart, A. (2011). Disruptor: High performance alternative to bounded queues for exchanging data between concurrent threads. Technical report, LMAX.

Appendix A

LockFreeQueue Implementation

This appendix contains the lock-free queue implementation used in the benchmark pipeline. The listing is provided in the submitted artefact bundle as source code under the corresponding implementation directories.

Appendix B

ArrayOrderBook addOrder

This appendix points to the focused ArrayOrderBook insertion path, including index mapping and best-level cache updates, as implemented in the submitted source artefacts.

Appendix C

Matching Engine Timestamp Decomposition

This appendix references the matching-engine processing path and timestamp decomposition used for network/queue/engine breakdown in the evaluation chapter.

Appendix D

FIX Incremental Parser

This appendix references the incremental FIX parser loop used for partial-message buffering and checksum boundary detection in the gateway path.

Appendix E

PoolOrderBook Match Branching

This appendix references the conditional order-filling and removal logic in the `PoolOrderBook::match` implementation (lines 131–156). Under mixed market workloads with varying order sizes, these branches result in the high misprediction rate (7.36%) documented in the evaluation.